

A Beginner's Tutorial on QSAR Modeling

A Practical Guide for Undergraduate Summer Research Students

Prepared by: *Amir Daghighi*

Email: *amirreza.daghighi@nds.u.edu*

LinkedIn: *<https://www.linkedin.com/in/amir-daghighi/>*

Computational Chemistry Lab / Department of Coatings and Polymeric Materials

May 2026

Abstract

This tutorial introduces undergraduate summer research students to Quantitative Structure–Activity Relationship (QSAR) modeling. It assumes no prior background in machine learning or computational chemistry. The tutorial is designed to be project-based: students learn the basic theory, then apply it using Python, RDKit, pandas, and scikit-learn. By the end of the tutorial, students should be able to clean a molecular dataset, calculate molecular descriptors and fingerprints, train simple QSAR models, evaluate model performance, understand applicability domain, and explain model predictions using chemical reasoning.

Contents

How to Use This Tutorial	4
1 What Is QSAR?	5
1.1 Basic Definition	5
1.2 Why QSAR Is Useful	6
1.3 General QSAR Workflow	6
2 Basics of Statistics and Metrics	7
2.1 Mean	7
2.2 Variance	8
2.3 Standard Deviation	9
2.4 Calculating Basic Statistics in Python	9
2.5 Regression Metrics in QSAR	9
2.6 Mean Absolute Error	10
2.7 Root Mean Squared Error	10
2.8 Coefficient of Determination: R^2	11
2.9 Calculating Regression Metrics in Python	12
2.10 Example Interpretation of Model Metrics	12
2.11 Summary	12
3 Basic Chemistry for QSAR	13
3.1 Structure Determines Properties	13
3.2 SMILES Notation	13
3.3 Visualizing Molecules with RDKit	15
4 Molecular Descriptors and Fingerprints	16
4.1 Classes of Molecular Descriptors	16
4.1.1 0D Descriptors	16
4.1.2 1D Descriptors	17
4.1.3 2D Descriptors	17
4.1.4 3D Descriptors	18
4.1.5 Quantum Chemical Descriptors	19
4.1.6 Electrostatic Descriptors	20
4.1.7 Summary of Descriptor Classes	20
4.1.8 Important Practical Considerations	21
4.2 Why We Need Descriptors (Features)	21
4.3 Simple Molecular Descriptor calculation using RDKit	21
4.4 Fingerprints	22
4.5 Descriptors vs. Fingerprints	22

5	Data Cleaning and Preprocessing	24
5.1	Removing Invalid SMILES	24
5.2	Removing Constant, Near-Constant, and Highly Correlated Descriptors	25
5.2.1	Removing Constant and Near-Constant Descriptors	25
5.2.2	Removing Highly Correlated Descriptors	26
5.3	Train and Test Split	27
5.3.1	Random Splitting	28
5.3.2	Sorted Splitting Based on the Target Variable	28
5.3.3	Cluster Splitting with UMAP	29
5.4	Summary	32
6	Machine Learning Basics for QSAR	34
6.1	What Is a Model?	34
6.2	Machine Learning vs. Rule-Based Systems	35
6.3	Regression vs. Classification	36
6.3.1	Regression	36
6.3.2	Classification	37
6.4	Overfitting and Underfitting	37
6.4.1	Underfitting	38
6.4.2	Overfitting	38
6.4.3	Good Generalization	38
6.5	Feature Selection	39
6.5.1	Why Feature Selection Is Important	39
6.5.2	Sequential Feature Selection	40
6.5.3	Genetic Algorithm-Based Feature Selection	41
6.5.4	Sequential Feature Selection vs. Genetic Algorithm	42
6.6	Summary	42
7	Internal and External Validation	43
7.1	Internal Validation	43
7.1.1	K-Fold Cross-Validation	44
7.1.2	Leave-One-Out Cross-Validation	45
7.1.3	Internal Validation Summary	47
7.2	External Validation	47
7.2.1	External Test Set Workflow	47
7.2.2	Why External Validation Is Important	48
7.2.3	Internal vs. External Validation	49
7.2.4	Common Mistakes	49
7.3	Summary	49
8	Applicability Domain	50
8.1	What Is Applicability Domain?	50
8.1.1	Why Applicability Domain Is Important	50
8.2	Approaches for Defining Applicability Domain	51
8.2.1	Distance-Based Applicability Domain	51
8.2.2	Similarity-Based Applicability Domain	52
8.2.3	Williams Plot	52
8.3	Understanding the Williams Plot	53
8.3.1	X-Axis: Leverage	53
8.3.2	Warning Leverage Threshold	53
8.3.3	Y-Axis: Standardized Residual	54
8.3.4	Meaning of ± 3 Standardized Residuals	55

8.4	Interpreting Regions of the Williams Plot	55
8.4.1	Inside vs. Outside Applicability Domain	56
8.5	Example Python Code for Williams Plot	56
8.6	Comparison of Applicability Domain Methods	58
8.7	Summary	58
9	Interpreting and Explaining a QSAR Model	60
9.1	Why Interpretability Is Important	60
9.2	Understanding Descriptor Effects	61
9.2.1	Positive Descriptor Effect	61
9.2.2	Negative Descriptor Effect	61
9.2.3	Nonlinear Descriptor Effect	62
9.2.4	Descriptor Effects Are Model-Dependent	62
9.3	Feature Importance	62
9.4	SHAP Values	63
9.4.1	Interpreting SHAP Values	63
9.5	Accumulated Local Effects	64
9.5.1	What Does an ALE Plot Show?	64
9.5.2	Interpreting ALE Plots	64
9.5.3	ALE vs. Simple Correlation	64
9.5.4	Example ALE Code	65
9.5.5	Using ALE for QSAR Interpretation	66
9.6	Important Cautions	66
9.7	Summary	66
A	Example Dataset Template (To be updated/placeholder)	67
B	Suggested Student Report Template	68
C	Useful Resources	69

How to Use This Tutorial

This document is designed as a flexible tutorial for undergraduate summer students. It can be taught over four to six weeks, depending on the student's background and the amount of time available.

Suggested Format

Each chapter contains:

- a short conceptual explanation,
- key terms,
- a coding or discussion exercise,
- suggested deliverables.

Recommended Timeline

Week	Main Topics
Week 1	Introduction to QSAR, SMILES, Python basics, RDKit visualization
Week 2	QSAR datasets, data cleaning, molecular descriptors and fingerprints
Week 3	Machine learning basics, regression and classification models
Week 4	Model evaluation, cross-validation, external validation, applicability domain
Week 5	Model interpretation, SHAP, descriptor meaning, chemical reasoning
Week 6	Final mini-project, written report, and short oral presentation

Instructor Note

For students with no programming background, spend more time on Chapters 2–4. For students with stronger coding skills, move faster through Python basics and focus more on validation, applicability domain, and interpretation.

Chapter 1

What Is QSAR?

Learning Objectives

- Define QSAR in simple terms.
- Explain why chemical structure can be used to predict biological or physicochemical activity.
- Recognize common QSAR endpoints such as toxicity, solubility, and binding affinity.
- Understand the general QSAR workflow.

1.1 Basic Definition

QSAR stands for **Quantitative Structure–Activity Relationship**. A QSAR model is a mathematical or machine-learning model that connects the chemical structure of a molecule to a measured property or activity. Figure below shows the general idea of what QSAR is.

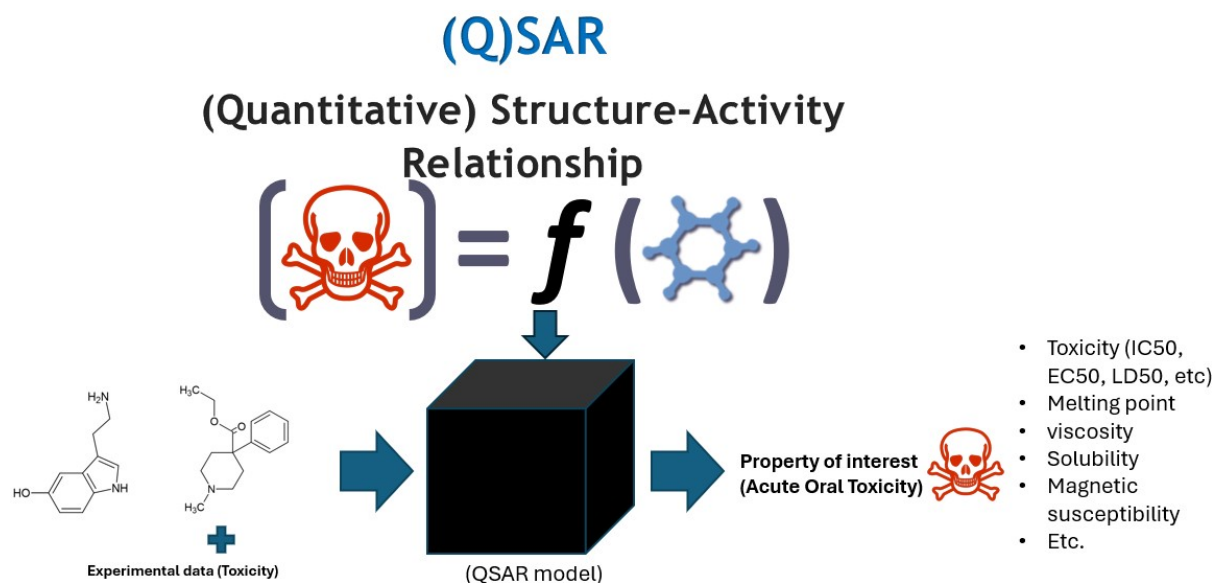


Figure 1.1: QSAR model is a function that relates structure of the compound to its activity. Our final goal will be to find this mapping (black box).

For example, a QSAR model may try to answer questions such as:

- Can we predict whether a compound is toxic?

- Can we estimate the solubility of a molecule?
- Can we predict binding affinity toward a protein target?
- Can we classify compounds as active or inactive?

1.2 Why QSAR Is Useful

Experimental testing can be expensive, slow, and sometimes ethically difficult (for example, toxicity tests on animals). QSAR can help researchers prioritize compounds before running experiments. It does not replace experiments, but it can guide decision-making.

1.3 General QSAR Workflow

A typical QSAR workflow includes:

1. Collect chemical structures and experimental activity values.
2. Clean and standardize the dataset.
3. Convert molecules into numerical features.
4. Split data into training and test sets.
5. Train machine-learning models.
6. Evaluate model performance.
7. Define applicability domain.
8. Interpret important chemical features.
9. Use the model to make predictions for new compounds.

Key Terms

- **Compound:** A chemical molecule in the dataset.
- **Endpoint:** The property or activity being predicted.
- **Descriptor:** A numerical value that describes a molecular property.
- **Model:** A mathematical function that maps descriptors to activity.
- **Prediction:** The model's estimated activity for a compound.

Chapter 2

Basics of Statistics and Metrics

Learning Objectives

- Understand mean, variance, and standard deviation.
- Explain why basic statistics are useful in QSAR.
- Understand common regression metrics used in QSAR modeling.
- Calculate and interpret MAE, RMSE, and R^2 .

Before evaluating QSAR models, it is important to understand a few basic statistical concepts. These concepts help us describe datasets, understand variation in experimental values, and evaluate how well a model predicts new compounds.

In QSAR modeling, we often compare experimental endpoint values, such as LD₅₀, IC₅₀, EC₅₀, or solubility, with predicted values generated by a machine-learning model. Model evaluation metrics help us measure the difference between experimental and predicted values.

2.1 Mean

The **mean**, also called the average, describes the central value of a dataset. It is calculated by adding all values together and dividing by the number of values.

For a set of values:

$$x_1, x_2, x_3, \dots, x_n$$

the mean is calculated as:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

where:

- $\bar{x}$ is the mean,
- x_i is each individual value,
- n is the total number of values.

Example

Suppose we have five toxicity values:

2.0, 3.0, 4.0, 5.0, 6.0

The mean is:

$$\bar{x} = \frac{2.0 + 3.0 + 4.0 + 5.0 + 6.0}{5} = 4.0$$

The mean gives a simple summary of the center of the data. However, it does not tell us how spread out the values are.

2.2 Variance

The **variance** describes how spread out the values are from the mean. If the values are close to the mean, the variance is small. If the values are far from the mean, the variance is large.

The sample variance is calculated as:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

where:

- s^2 is the sample variance,
- x_i is each value,
- $\bar{x}$ is the mean,
- n is the number of values.

The term $(x_i - \bar{x})$ measures how far each value is from the mean. This difference is squared so that positive and negative differences do not cancel each other out.

Example

Using the same values:

2.0, 3.0, 4.0, 5.0, 6.0

The mean is:

$$\bar{x} = 4.0$$

The deviations from the mean are:

-2.0, -1.0, 0.0, 1.0, 2.0

The squared deviations are:

4.0, 1.0, 0.0, 1.0, 4.0

The sample variance is:

$$s^2 = \frac{4.0 + 1.0 + 0.0 + 1.0 + 4.0}{5 - 1}$$
$$s^2 = \frac{10.0}{4} = 2.5$$

2.3 Standard Deviation

The **standard deviation** is the square root of the variance. It is often easier to interpret than variance because it has the same unit as the original data.

$$s = \sqrt{s^2}$$

For the previous example:

$$s = \sqrt{2.5} = 1.58$$

The standard deviation tells us how much the values typically vary around the mean.

Interpretation

Standard Deviation	Meaning
Small standard deviation	Values are close to the mean
Large standard deviation	Values are spread far from the mean

In QSAR, standard deviation can help describe the spread of experimental endpoint values. For example, if the toxicity values cover a wide range, the dataset may be easier to model than a dataset where all values are very similar.

2.4 Calculating Basic Statistics in Python

The following code shows how to calculate mean, variance, and standard deviation in Python.

Listing 2.1: Calculating mean, variance, and standard deviation.

```
1 import numpy as np
2
3 values = np.array([2.0, 3.0, 4.0, 5.0, 6.0])
4
5 mean_value = np.mean(values)
6 variance_value = np.var(values, ddof=1)      # sample variance
7 std_value = np.std(values, ddof=1)           # sample standard deviation
8
9 print("Mean:", mean_value)
10 print("Variance:", variance_value)
11 print("Standard deviation:", std_value)
```

The argument `ddof=1` is used to calculate the sample variance and sample standard deviation. This means the denominator is $n - 1$ instead of n .

2.5 Regression Metrics in QSAR

After training a QSAR regression model, we need to compare the predicted values with the experimental values. This helps us understand how accurate the model is.

Common regression metrics include:

- Mean Absolute Error, or MAE,
- Root Mean Squared Error, or RMSE,
- coefficient of determination, or R^2 .

Suppose:

- y_i is the experimental value for compound i ,
- $\hat{y}_i$ is the predicted value for compound i ,
- n is the number of compounds.

The prediction error for compound i is:

$$e_i = y_i - \hat{y}_i$$

A perfect model would have prediction errors close to zero for all compounds.

2.6 Mean Absolute Error

The **Mean Absolute Error**, abbreviated as **MAE**, is the average absolute difference between experimental and predicted values.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

MAE tells us the average size of the prediction error.

Interpretation

If a toxicity model has:

$$MAE = 0.25$$

this means that, on average, the model predictions are off by about 0.25 units of the endpoint.
MAE is easy to interpret because it is in the same unit as the target variable.

Advantages of MAE

- Easy to understand.
- Less sensitive to very large errors than RMSE.
- Same unit as the endpoint.

2.7 Root Mean Squared Error

The **Root Mean Squared Error**, abbreviated as **RMSE**, measures the square root of the average squared prediction error.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Because the errors are squared before averaging, RMSE gives more weight to large errors. This means RMSE is more sensitive to outliers than MAE.

Interpretation

If a model has:

$$RMSE = 0.40$$

this means that the typical prediction error is about 0.40 units, with larger errors being penalized more strongly.

MAE vs. RMSE

Metric	Meaning	Sensitivity to Large Errors
MAE	Average absolute error	Lower
RMSE	Square-root average squared error	Higher

If RMSE is much larger than MAE, it may indicate that the model has a few compounds with very large prediction errors.

2.8 Coefficient of Determination: R^2

The coefficient of determination, written as R^2 , measures how much of the variation in the experimental values is explained by the model.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

where:

- y_i is the experimental value,
- $\hat{y}_i$ is the predicted value,
- $\bar{y}$ is the mean of the experimental values,
- n is the number of compounds.

The numerator measures the model prediction error. The denominator measures the total variation in the experimental data.

Interpretation of R^2

R^2 Value	Interpretation
$R^2 = 1$	Perfect prediction
$R^2 = 0$	Model performs similar to predicting the mean
$R^2 < 0$	Model performs worse than predicting the mean

A higher R^2 generally indicates better model performance. However, R^2 should not be used alone. A model can have a high R^2 but still have large errors for some compounds.

Important Note About R^2

R^2 can be sensitive to the range and distribution of the endpoint values. If the dataset covers a very wide range of activity values, R^2 may appear high even when some prediction errors are large. If the dataset has a very narrow activity range, R^2 may appear low even when the absolute errors are small.

For this reason, QSAR models should usually report R^2 together with error-based metrics such as MAE and RMSE.

2.9 Calculating Regression Metrics in Python

The following code shows how to calculate MAE, RMSE, and R^2 using scikit-learn.

Listing 2.2: Calculating MAE, RMSE, and R2 in Python.

```
1 from sklearn.metrics import mean_absolute_error
2 from sklearn.metrics import mean_squared_error
3 from sklearn.metrics import r2_score
4 import numpy as np
5
6 # Example experimental and predicted values
7 y_true = np.array([2.0, 3.0, 4.0, 5.0, 6.0])
8 y_pred = np.array([2.2, 2.8, 4.1, 4.7, 6.3])
9
10 mae = mean_absolute_error(y_true, y_pred)
11 rmse = np.sqrt(mean_squared_error(y_true, y_pred))
12 r2 = r2_score(y_true, y_pred)
13
14 print("MAE:", mae)
15 print("RMSE:", rmse)
16 print("R2:", r2)
```

2.10 Example Interpretation of Model Metrics

Suppose a QSAR model gives the following external test results:

Metric	Value
MAE	0.22
RMSE	0.35
R^2	0.86

This means that the model explains a large portion of the variation in the experimental data because $R^2 = 0.86$. The average absolute prediction error is 0.22 units. The RMSE is larger than the MAE, which may suggest that a few compounds have larger prediction errors.

A good model evaluation should not rely on only one metric. MAE, RMSE, and R^2 should be interpreted together.

2.11 Summary

Mean, variance, and standard deviation are basic statistics used to describe data. The mean describes the central value, while variance and standard deviation describe how spread out the values are.

In QSAR regression modeling, MAE, RMSE, and R^2 are commonly used to evaluate model performance. MAE measures the average absolute prediction error. RMSE also measures prediction error but gives more weight to large errors. R^2 measures how much variation in the experimental data is explained by the model.

A reliable QSAR model should report multiple metrics because each metric describes model performance from a different perspective.

Chapter 3

Basic Chemistry for QSAR

Learning Objectives

- Understand why molecular structure affects activity.
- Recognize SMILES strings as text representations of molecules.
- Identify common molecular properties used in QSAR.

3.1 Structure Determines Properties

The core idea of QSAR is that we assume the structure of a molecule contains information about its activity. For example, molecular size, polarity, charge, hydrophobicity, hydrogen bonding, and functional groups can all influence how a molecule behaves.

3.2 SMILES Notation

SMILES stands for **Simplified Molecular Input Line Entry System**. It is a text-based way to represent a chemical structure. In QSAR modeling, SMILES strings are commonly used because they allow molecules to be stored in a simple table format and processed by cheminformatics tools such as RDKit.

For example, ethanol can be written as:

CCO

This means that ethanol contains two carbon atoms followed by one oxygen atom. Although SMILES strings may look simple, they follow specific rules to describe atoms, bonds, rings, branches, charges, and stereochemistry.

Common SMILES Rules

Rule	Example	Meaning
Atoms are written using atomic symbols	C, O, N, Cl	Carbon, oxygen, nitrogen, chlorine
Single bonds are often omitted	CCO	Ethanol: carbon-carbon-oxygen
Double bonds use =	C=O	Carbonyl group
Triple bonds use #	C#N	Nitrile group
Branches use parentheses	CC(C)O	A branched alcohol
Rings use numbers	C1CCCCC1	Cyclohexane ring
Aromatic atoms are lowercase	c1ccccc1	Benzene ring
Charged atoms use brackets	[NH4+]	Ammonium ion
Disconnected fragments use a period	Na.Cl or [Na+] . [Cl-]	Separate ionic components
Stereochemistry can be specified	F[C@H](Cl)Br	Chiral center

Examples

Table below shows examples of some common small molecules. You can find a visualization of them in Figure 2.1

Molecule	SMILES	Description
Methane	C	One carbon atom
Ethanol	CCO	Two carbons and one hydroxyl group
Acetic acid	CC(=O)O	Contains a carboxylic acid group
Cyclohexane	C1CCCCC1	Six-membered carbon ring
Benzene	c1ccccc1	Aromatic six-membered ring
Phenol	c1ccccc1O	Benzene ring with hydroxyl group

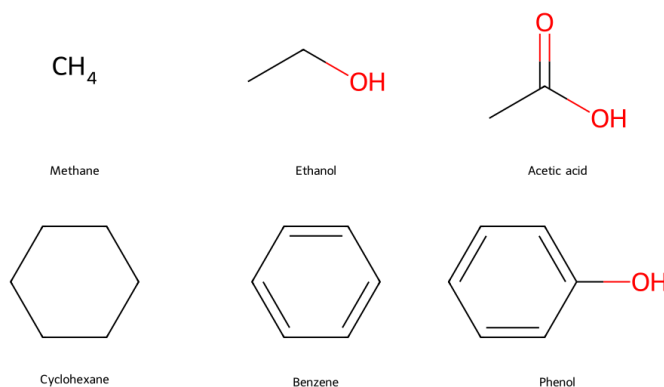


Figure 3.1: Examples of molecular structures generated from SMILES strings using RDKit. The molecule names are shown below each structure.

Important Notes

A single molecule can sometimes be written using more than one valid SMILES string. For example, the order in which atoms are written may change, but the chemical structure can still represent the same molecule. To avoid this issue, cheminformatics software can generate a **canonical SMILES**, which is a standardized version of the SMILES string.

For QSAR modeling, SMILES strings are important because they are the starting point for generating molecular descriptors and fingerprints. However, before modeling, SMILES strings should be checked carefully because invalid or inconsistent structures can lead to errors in descriptor calculation.

Exercise

Look at the following SMILES strings and identify whether each molecule contains a ring, a branch, a double bond, or an aromatic system:

1. CCO
2. CC(=O)O
3. C1CCCCC1
4. c1ccccc1
5. CC(C)O

3.3 Visualizing Molecules with RDKit

Listing 3.1: Visualizing a molecule from SMILES using RDKit.

```
1 from rdkit import Chem
2 from rdkit.Chem import Draw
3
4 smiles = "CCO" # ethanol
5 mol = Chem.MolFromSmiles(smiles)
6
7 Draw.MolToImage(mol)
```

Exercise

visualize each molecule in the previous exercise using RDKit and identify one visible structural feature, such as a ring, oxygen atom, or carboxylic acid group.

Chapter 4

Molecular Descriptors and Fingerprints

Learning Objectives

- Explain why molecules must be converted into numerical features.
- Calculate simple RDKit descriptors.
- Understand the difference between descriptors and fingerprints.

4.1 Classes of Molecular Descriptors

Molecular descriptors are numerical values that represent different aspects of a molecule. These descriptors allow chemical structures to be converted into a format that machine-learning models can use. Depending on the type of structural information used, molecular descriptors are often grouped into several classes, including 0D, 1D, 2D, 3D, quantum chemical, and electrostatic descriptors.

In general, lower-dimensional descriptors are easier to calculate and interpret, while higher-dimensional and quantum-based descriptors may capture more detailed structural and electronic information but often require more computational effort.

4.1.1 0D Descriptors

Zero-dimensional, or **0D descriptors**, describe the overall composition of a molecule without considering atom connectivity or molecular geometry. These descriptors are usually simple counts or global molecular properties.

Examples of 0D descriptors include:

- molecular weight,
- number of atoms,
- number of carbon atoms,
- number of heteroatoms,
- number of hydrogen bond donors,
- number of hydrogen bond acceptors,

- number of rings.

For example, molecular weight gives a simple measure of molecular size. The number of hydrogen bond donors and acceptors provides information about the molecule’s ability to form hydrogen bonds, which can influence solubility, permeability, and biological activity.

Example Descriptor	Chemical Meaning
Molecular weight	Overall molecular size
Number of heteroatoms	Presence of non-carbon atoms such as O, N, S, or halogens
Number of H-bond donors	Ability to donate hydrogen bonds
Number of H-bond acceptors	Ability to accept hydrogen bonds
Number of rings	Ring content and structural rigidity

4.1.2 1D Descriptors

One-dimensional, or **1D descriptors**, describe molecular features based on lists or counts of specific fragments, atoms, or functional groups. They provide more chemical detail than 0D descriptors but still do not require full molecular geometry.

Examples of 1D descriptors include:

- count of hydroxyl groups,
- count of carboxylic acid groups,
- count of aromatic atoms,
- count of halogen atoms,
- count of specific atom types or functional groups.

These descriptors are useful because the presence or absence of certain functional groups can strongly influence molecular activity. For example, carboxylic acid groups may increase polarity and ionization, while aromatic rings may contribute to hydrophobic interactions or π – π stacking.

Example Descriptor	Chemical Meaning
Number of hydroxyl groups	Hydrogen bonding and polarity
Number of carboxylic acid groups	Acidity, ionization, and polarity
Number of aromatic atoms	Aromatic character and possible π interactions
Number of halogen atoms	Hydrophobicity and electronic effects
Functional group counts	Presence of chemically important fragments

4.1.3 2D Descriptors

Two-dimensional, or **2D descriptors**, use the molecular graph, meaning the atoms and bonds in the molecule. These descriptors consider how atoms are connected, but they do not require a 3D molecular conformation.

Examples of 2D descriptors include:

- topological polar surface area,
- molecular connectivity indices,
- topological autocorrelation descriptors,

- path counts,
- Kier–Hall indices,
- fragment-based descriptors,
- 2D fingerprints such as Morgan fingerprints.

2D descriptors are widely used in QSAR because they are relatively fast to calculate and often provide strong predictive performance. They can capture important structural patterns such as branching, ring systems, atom connectivity, and functional group arrangement.

For example, topological polar surface area, often abbreviated as TPSA, estimates the polar surface area of a molecule from its 2D structure. TPSA is frequently related to solubility, membrane permeability, and oral bioavailability.

Example Descriptor	Chemical Meaning
TPSA	Polar surface area estimated from 2D structure
Connectivity indices	Molecular branching and atom connectivity
Topological autocorrelations	Distribution of atomic properties across molecular graph distances
Fragment descriptors	Presence of specific structural fragments
Morgan fingerprints	Local atomic environments encoded as bits

4.1.4 3D Descriptors

Three-dimensional, or **3D descriptors**, describe the spatial arrangement of atoms in a molecule. Unlike 0D, 1D, and 2D descriptors, 3D descriptors require a molecular conformation.

Examples of 3D descriptors include:

- molecular volume,
- solvent-accessible surface area,
- 3D polar surface area,
- radius of gyration,
- molecular shape descriptors,
- WHIM descriptors,
- GETAWAY descriptors,
- 3D autocorrelation descriptors.

3D descriptors can be important when biological activity depends on molecular shape, steric effects, or the spatial arrangement of functional groups. For example, two molecules may have similar atom connectivity but different 3D conformations, leading to different interactions with a biological target.

However, 3D descriptors require careful attention because flexible molecules can adopt multiple conformations. The selected conformation can affect the descriptor values and, therefore, the model predictions.

Example Descriptor	Chemical Meaning
Molecular volume	Overall 3D size of the molecule
Surface area	Molecular exposure to surrounding environment
Radius of gyration	Molecular compactness or spread
WHIM descriptors	Molecular size, shape, symmetry, and atom distribution
GETAWAY descriptors	Geometry, topology, and atom weighting information

4.1.5 Quantum Chemical Descriptors

Quantum chemical descriptors are calculated using quantum mechanics-based methods. These descriptors describe the electronic structure of a molecule and can provide information about reactivity, stability, and electron distribution.

Examples of quantum chemical descriptors include:

- HOMO energy,
- LUMO energy,
- HOMO–LUMO gap,
- dipole moment,
- polarizability,
- partial atomic charges,
- ionization potential,
- electron affinity,
- molecular electrostatic potential.

The **HOMO**, or highest occupied molecular orbital, is related to the molecule’s ability to donate electrons. The **LUMO**, or lowest unoccupied molecular orbital, is related to the molecule’s ability to accept electrons. The HOMO–LUMO gap can provide information about molecular stability and chemical reactivity.

Quantum chemical descriptors are often useful when the endpoint is influenced by electronic effects, such as redox activity, photoluminescence, chemical reactivity, or interactions involving charge transfer. However, these descriptors are usually more computationally expensive than simple 2D descriptors.

Example Descriptor	Chemical Meaning
HOMO energy	Electron-donating ability
LUMO energy	Electron-accepting ability
HOMO–LUMO gap	Molecular stability and reactivity
Dipole moment	Overall molecular polarity
Polarizability	Ability of electron cloud to be distorted
Partial charges	Local electron distribution around atoms

4.1.6 Electrostatic Descriptors

Electrostatic descriptors describe the distribution of charge and electrostatic potential across a molecule. These descriptors are especially important when molecular interactions are influenced by polarity, charge, hydrogen bonding, or ionic interactions.

Examples of electrostatic descriptors include:

- partial atomic charges,
- maximum positive charge,
- maximum negative charge,
- total positive charge,
- total negative charge,
- molecular dipole moment,
- electrostatic potential surface descriptors,
- charge-weighted autocorrelation descriptors.

Electrostatic descriptors can help describe how a molecule interacts with proteins, membranes, solvents, or other molecules. For example, a molecule with a strong positive region may interact with negatively charged amino acid residues in a protein binding site. Similarly, charge distribution can influence toxicity, solubility, and membrane transport.

Example Descriptor	Chemical Meaning
Partial atomic charges	Local charge distribution
Maximum positive charge	Most electron-deficient region
Maximum negative charge	Most electron-rich region
Dipole moment	Separation of positive and negative charge
Electrostatic potential	Charge pattern on molecular surface
Charge autocorrelation	Distribution of charge across molecular distances

4.1.7 Summary of Descriptor Classes

Class	Main Information Captured	Example Descriptors
0D	Molecular composition and simple counts	Molecular weight, atom counts, ring count
1D	Functional groups and atom-type counts	Hydroxyl count, halogen count, aromatic atom count
2D	Molecular graph and atom connectivity	TPSA, connectivity indices, fragment descriptors, fingerprints
3D	Molecular geometry and shape	Molecular volume, surface area, WHIM, GETAWAY
Quantum chemical	Electronic structure and reactivity	HOMO, LUMO, HOMO-LUMO gap, polarizability
Electrostatic	Charge distribution and polarity	Partial charges, dipole moment, electrostatic potential

4.1.8 Important Practical Considerations

The choice of descriptor depends on the QSAR problem, dataset size, available computational resources, and the type of chemical information expected to influence the endpoint.

For beginner QSAR projects, up to 2D descriptors are often a good starting point because they are easy to calculate, interpretable, and usually perform well for many endpoints. For more advanced projects, 3D, quantum chemical, and electrostatic descriptors may provide additional information, especially when molecular shape, conformation, or electronic structure plays an important role. This could be the case for protein-ligand QSAR models that need to consider the ligand's pose and the protein pocket in 3D space.

4.2 Why We Need Descriptors (Features)

Machine-learning models cannot directly understand chemical drawings or SMILES strings. We must convert each molecule into numerical values.

These values are called **features**. In QSAR, features are often molecular descriptors or fingerprints.

4.3 Simple Molecular Descriptor calculation using RDKit

Use the following code to practice generating certain molecular descriptors using RDKit package for the ibuprofen molecule.

Listing 4.1: Calculating simple RDKit descriptors.

```
1 from rdkit import Chem
2 from rdkit.Chem import Descriptors
3
4 smiles = "CC(C)Cc1ccc(cc1)C(C)C(=O)O" # ibuprofen
5 mol = Chem.MolFromSmiles(smiles)
6
7 features = {
8     "MolWt": Descriptors.MolWt(mol),
9     "MolLogP": Descriptors.MolLogP(mol),
10    "TPSA": Descriptors.TPSA(mol),
11    "NumHDonors": Descriptors.NumHDonors(mol),
12    "NumHAcceptors": Descriptors.NumHAcceptors(mol),
13    "NumRotatableBonds": Descriptors.NumRotatableBonds(mol)
14 }
15
16 print(features)
```

You can also calculate all the descriptors available on the RDKit package using the code below:

Listing 4.2: Calculating all default RDKit descriptors for ibuprofen.

```
1 from rdkit import Chem
2 from rdkit.Chem import Descriptors
3 import pandas as pd
4 import numpy as np
5
6 # Ibuprofen SMILES
7 smiles = "CC(C)Cc1ccc(cc1)C(C)C(=O)O"
8
9 # Convert SMILES to RDKit molecule object
10 mol = Chem.MolFromSmiles(smiles)
```

```

11
12 # Check that the SMILES was successfully converted
13 if mol is None:
14     raise ValueError("Invalid SMILES string.")
15
16 # Calculate all default RDKit descriptors
17 all_descriptors = Descriptors.CalcMolDescriptors(
18     mol,
19     missingVal=np.nan
20 )
21
22 # Convert dictionary to a DataFrame for easier viewing
23 descriptor_table = pd.DataFrame(
24     all_descriptors.items(),
25     columns=["Descriptor", "Value"]
26 )
27
28 print("Number of descriptors calculated:", len(descriptor_table))
29 print(descriptor_table.head(10))
30
31 # Optional: save all descriptors to a CSV file
32 descriptor_table.to_csv("ibuprofen_rdkit_descriptors.csv", index=False)

```

4.4 Fingerprints

Fingerprints encode the presence or absence of molecular substructures. One common fingerprint is the Morgan fingerprint, also known as ECFP-like fingerprint.

Listing 4.3: Calculating a Morgan fingerprint.

```

1 from rdkit import Chem
2 from rdkit.Chem import AllChem
3 import numpy as np
4
5 smiles = "CCO"
6 mol = Chem.MolFromSmiles(smiles)
7
8 fp = AllChem.GetMorganFingerprintAsBitVect(mol, radius=2, nBits=1024)
9
10 arr = np.zeros((1024,), dtype=int)
11 Chem.DataStructs.ConvertToNumpyArray(fp, arr)
12
13 print(arr[:20])
14 print("Fingerprint length:", len(arr))

```

4.5 Descriptors vs. Fingerprints

Feature Type	Advantage	Limitation
Descriptors	Often interpretable	May miss structural details
Fingerprints	Good for structural similarity	Less directly interpretable

Exercise

Ask the student to calculate RDKit descriptors for 10 molecules and rank the molecules by molecular weight, logP, and TPSA. Then ask: which molecule would you expect to be more hydrophobic?

Chapter 5

Data Cleaning and Preprocessing

Learning Objectives

- Remove invalid SMILES before descriptor calculation.
- Remove constant, near-constant, and highly correlated descriptors.
- Understand why train/test splitting is necessary in QSAR.
- Compare random splitting, sorted splitting, and cluster splitting.

Before building a QSAR model, the dataset must be cleaned and prepared carefully. Data preprocessing is one of the most important steps in QSAR modeling because poor-quality chemical structures, unreliable descriptors, duplicate information, or inappropriate train/test splitting can lead to misleading model performance.

In a typical QSAR workflow, preprocessing includes checking molecular structures, calculating descriptors, removing problematic descriptor columns, and splitting the data into training and test sets.

5.1 Removing Invalid SMILES

SMILES strings are used to represent chemical structures in a text format. However, not every SMILES string in a dataset is necessarily valid. A SMILES may be invalid because of formatting errors, missing atoms, incorrect bonding, or other structural problems.

Before calculating descriptors, each SMILES string should be converted into an RDKit molecule object. If RDKit cannot interpret the SMILES, the molecule should be removed or corrected before modeling.

Listing 5.1: Removing invalid SMILES using RDKit.

```
1 from rdkit import Chem
2 import pandas as pd
3
4 # Example: load a dataset containing a SMILES column and an activity
   column
5 df = pd.read_csv("example_qsar_dataset.csv")
6
7 # Function to convert SMILES into RDKit molecule objects
8 def smiles_to_mol(smiles):
9     try:
10         return Chem.MolFromSmiles(smiles)
11     except:
12         return None
```

```

13
14 # Convert SMILES to molecules
15 df["Mol"] = df["SMILES"].apply(smiles_to_mol)
16
17 # Count invalid SMILES
18 num_invalid = df["Mol"].isna().sum()
19 print("Number of invalid SMILES:", num_invalid)
20
21 # Remove invalid molecules
22 df = df.dropna(subset=["Mol"]).copy()
23
24 print("Dataset size after removing invalid SMILES:", df.shape)

```

After this step, only molecules that RDKit can successfully read are kept in the dataset. This is important because invalid SMILES will cause errors during descriptor calculation.

5.2 Removing Constant, Near-Constant, and Highly Correlated Descriptors

After molecular descriptors are calculated, the descriptor table may contain hundreds or thousands of columns. Not all descriptors are useful for modeling. Some descriptors may contain the same value for every compound, some may vary only slightly, and others may be highly correlated with each other.

Removing these descriptors helps reduce noise, improve model stability, and avoid unnecessary complexity.

5.2.1 Removing Constant and Near-Constant Descriptors

A **constant descriptor** has the same value for every molecule in the dataset. For example, if a descriptor has the value zero for every compound, it cannot help the model distinguish one compound from another.

A **near-constant descriptor** has almost the same value for most compounds. These descriptors usually provide very little useful information and may be removed before modeling.

Listing 5.2: Removing constant and near-constant descriptors.

```

1 from sklearn.feature_selection import VarianceThreshold
2 import pandas as pd
3
4 # X is the descriptor matrix
5 # y is the target variable
6 # Example:
7 # X = df[descriptor_columns]
8 # y = df["Activity"]
9
10 # Remove descriptors with missing values, if needed
11 X = X.dropna(axis=1)
12
13 # Remove constant and near-constant descriptors
14 # A threshold of 0 removes only perfectly constant descriptors.
15 # A small positive threshold removes near-constant descriptors.
16 selector = VarianceThreshold(threshold=1e-8)
17
18 X_variance_filtered_array = selector.fit_transform(X)
19
20 # Keep the names of selected descriptors

```

```

21 selected_columns = X.columns[selector.get_support()]
22
23 X_variance_filtered = pd.DataFrame(
24     X_variance_filtered_array,
25     columns=selected_columns,
26     index=X.index
27 )
28
29 print("Original number of descriptors:", X.shape[1])
30 print("Number after removing constant/near-constant descriptors:",
31       X_variance_filtered.shape[1])

```

5.2.2 Removing Highly Correlated Descriptors

Many molecular descriptors measure similar chemical information. For example, molecular weight and atom count are often strongly related. If two descriptors are highly correlated, they may provide redundant information.

A common approach is to calculate the Pearson correlation coefficient between descriptor pairs and remove one descriptor from each pair where the absolute correlation is greater than a selected threshold, such as $|R| > 0.95$.

Important: Highly correlated descriptors should be identified using only the **training set** (See section 6.3 to understand what the training set and test set mean). After the correlated descriptors are removed from the training set, the same descriptor columns should be removed from the test set. The correlation matrix should **not** be calculated using the combined training and test data, because this allows information from the test set to influence preprocessing decisions. This is a form of **data leakage**.

Data leakage occurs when information from the test set is accidentally used during model training or preprocessing. If this happens, the test set is no longer a truly independent evaluation set, and the reported model performance may be overly optimistic.

Listing 5.3: Removing highly correlated descriptors using only the training set.

```

1  import numpy as np
2  import pandas as pd
3  from sklearn.model_selection import train_test_split
4
5  # X is the descriptor matrix
6  # y is the target variable
7
8  # First split the data into training and test sets
9  X_train, X_test, y_train, y_test = train_test_split(
10     X_variance_filtered,
11     y,
12     test_size=0.2,
13     random_state=42
14 )
15
16 def find_highly_correlated_features(X_train, threshold=0.95):
17     """
18     Identify highly correlated descriptors using only the training set.
19     One descriptor is removed from each pair with absolute Pearson
20     correlation greater than the threshold.
21     """
22
23     # Calculate absolute correlation matrix using training data only
24     corr_matrix = X_train.corr().abs()

```

```

25
26     # Keep only the upper triangle of the correlation matrix
27     upper_triangle = corr_matrix.where(
28         np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
29     )
30
31     # Find descriptors to remove
32     columns_to_drop = [
33         column for column in upper_triangle.columns
34         if any(upper_triangle[column] > threshold)
35     ]
36
37     return columns_to_drop
38
39 # Identify correlated descriptors from training set only
40 columns_to_drop = find_highly_correlated_features(
41     X_train,
42     threshold=0.95
43 )
44
45 # Remove the same descriptors from both training and test sets
46 X_train_filtered = X_train.drop(columns=columns_to_drop)
47 X_test_filtered = X_test.drop(columns=columns_to_drop)
48
49 print("Number of descriptors before correlation filtering:",
50       X_train.shape[1])
51
52 print("Number of descriptors after correlation filtering:",
53       X_train_filtered.shape[1])
54
55 print("Number of descriptors removed:",
56       len(columns_to_drop))

```

This approach keeps the test set independent. The model is trained using descriptors selected from the training data only, and the test set is used only for final evaluation.

Important note: A common mistake is to calculate descriptor correlations using the full dataset before train/test splitting. Although this may seem harmless because the target variable is not used, it still allows the test set to influence feature selection. In proper QSAR modeling, preprocessing decisions such as scaling, feature filtering, descriptor selection, and hyperparameter tuning should be learned from the training set and then applied to the test set.

5.3 Train and Test Split

After the dataset is cleaned and descriptors are prepared, the next step is to split the dataset into a **training set** and a **test set**.

The training set is used to build the model. The test set is kept separate and is used only at the end to evaluate how well the model performs on new compounds.

This is important because a QSAR model should not only perform well on molecules it has already seen. The goal is to make reliable predictions for new molecules.

Training set → used to train the model

Test set → used to evaluate prediction performance

If the model performs very well on the training set but poorly on the test set, this may indicate **overfitting**. Overfitting means the model has learned patterns specific to the training data but does not generalize well to new compounds.

Common train/test splitting strategies include:

- random splitting,
- sorted splitting based on the target variable,
- cluster splitting based on chemical space.

5.3.1 Random Splitting

In a random split, compounds are randomly assigned to the training and test sets. For example, 80% of the data may be used for training and 20% for testing.

Random splitting is simple and commonly used. However, in QSAR, it can sometimes give overly optimistic results if very similar compounds appear in both the training and test sets.

Listing 5.4: Random train/test split.

```
1 from sklearn.model_selection import train_test_split
2
3 # X_filtered is the final descriptor matrix
4 # y is the target variable
5
6 X_train, X_test, y_train, y_test = train_test_split(
7     X_filtered,
8     y,
9     test_size=0.2,
10    random_state=42
11 )
12
13 print("Training set size:", X_train.shape[0])
14 print("Test set size:", X_test.shape[0])
```

Random splitting is useful for beginner projects because it is easy to implement and understand. However, it should not always be assumed to represent true external prediction performance.

5.3.2 Sorted Splitting Based on the Target Variable

In a sorted split, the dataset is first sorted by the target variable, such as toxicity, solubility, or IC₅₀. Then compounds are selected systematically for the test set across the full range of activity values. For example, if we can sort the data ascending or descending and put every 5th compound in the test set to get a 75%/25% train/test split. Figure 4.1 shows a visualization of the data distribution for training and testing after a sorted split.

This method helps make sure that the training and test sets cover a similar range of endpoint values. For example, if the endpoint is toxicity, the test set should ideally include low-toxicity, medium-toxicity, and high-toxicity compounds.

Listing 5.5: Sorted split based on the target variable.

```
1 import numpy as np
2
3 # Combine X and y temporarily so they can be sorted together
4 data_for_split = X_filtered.copy()
5 data_for_split["target"] = y
6
```

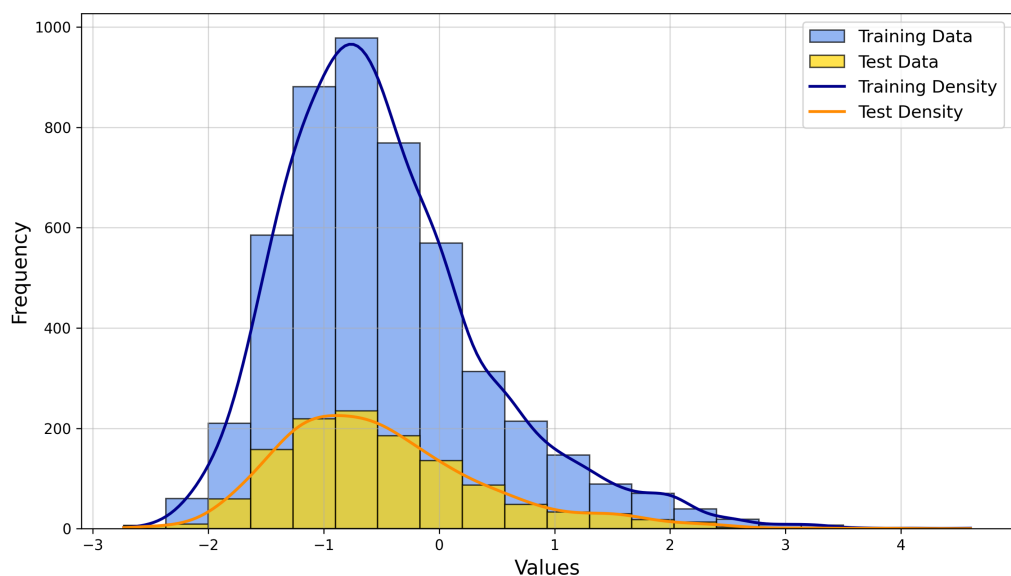


Figure 5.1: Sorted splitting guarantees a similar distribution of endpoint value in the train and test set.

```

7 # Sort by the target variable
8 data_for_split = data_for_split.sort_values(by="target").reset_index(
    drop=True)
9
10 # Select every 5th compound for the test set
11 # This gives approximately a 20% test set
12 test_indices = np.arange(len(data_for_split)) % 5 == 0
13
14 test_data = data_for_split[test_indices]
15 train_data = data_for_split[~test_indices]
16
17 # Separate features and target again
18 X_train = train_data.drop(columns=["target"])
19 y_train = train_data["target"]
20
21 X_test = test_data.drop(columns=["target"])
22 y_test = test_data["target"]
23
24 print("Training set size:", X_train.shape[0])
25 print("Test set size:", X_test.shape[0])
26
27 print("Training target range:", y_train.min(), "to", y_train.max())
28 print("Test target range:", y_test.min(), "to", y_test.max())

```

Sorted splitting can be useful when the dataset is small and the researcher wants both the training and test sets to cover the full activity range. However, it still does not directly ensure that the training and test sets are chemically diverse.

5.3.3 Cluster Splitting with UMAP

In QSAR, molecules are not just numbers; they occupy a chemical space. Some molecules are structurally similar, while others are very different. A cluster-based split tries to account for this by grouping molecules according to descriptor similarity.

One practical approach is to use **UMAP**, which stands for Uniform Manifold Approximation

and Projection. UMAP reduces high-dimensional descriptor data into a smaller number of dimensions, often two dimensions. Then, clustering can be performed in the UMAP space.

A UMAP-based cluster split can be useful because it considers the distribution of molecules in chemical space rather than only randomly splitting rows in a table.

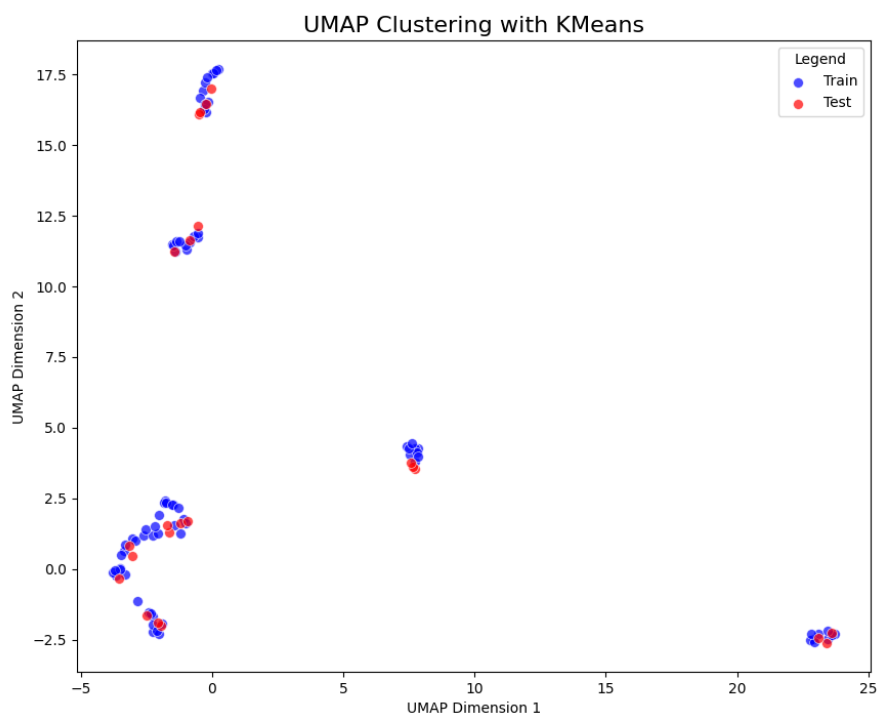


Figure 5.2: Example of using UMAP clustering to split data into train and test. Each cluster tends to contain structurally similar compounds, and within each cluster, we split the data randomly into train and test using desired ratio.

Basic Idea

The general workflow is:

1. Start with the molecular descriptor matrix.
2. Scale the descriptors.
3. Use UMAP to project the descriptors into two dimensions.
4. Cluster the molecules in the UMAP space.
5. Select training and test compounds from each cluster.

This approach helps the training and test sets represent different regions of chemical space.

Listing 5.6: UMAP-based cluster split.

```
1 import numpy as np
2 import pandas as pd
3
4 from sklearn.preprocessing import StandardScaler
```

```

5 from sklearn.cluster import KMeans
6 from sklearn.model_selection import train_test_split
7
8 import umap
9
10 # Scale descriptors before UMAP
11 scaler = StandardScaler()
12 X_scaled = scaler.fit_transform(X_filtered)
13
14 # Reduce descriptor space to 2 dimensions using UMAP
15 umap_model = umap.UMAP(
16     n_components=2,
17     random_state=42
18 )
19
20 X_umap = umap_model.fit_transform(X_scaled)
21
22 # Cluster molecules in UMAP space
23 n_clusters = 8
24
25 kmeans = KMeans(
26     n_clusters=n_clusters,
27     random_state=42,
28     n_init=10
29 )
30
31 cluster_labels = kmeans.fit_predict(X_umap)
32
33 # Create a DataFrame to store cluster information
34 split_df = pd.DataFrame({
35     "index": X_filtered.index,
36     "cluster": cluster_labels,
37     "target": y.values
38 })
39
40 train_indices = []
41 test_indices = []
42
43 # Sample training and test compounds from each cluster
44 for cluster_id in sorted(split_df["cluster"].unique()):
45
46     cluster_data = split_df[split_df["cluster"] == cluster_id]
47
48     # If a cluster is very small, keep all molecules in training set
49     if len(cluster_data) < 3:
50         train_indices.extend(cluster_data["index"].tolist())
51         continue
52
53     cluster_train, cluster_test = train_test_split(
54         cluster_data,
55         test_size=0.2,
56         random_state=42
57     )
58
59     train_indices.extend(cluster_train["index"].tolist())
60     test_indices.extend(cluster_test["index"].tolist())
61
62 # Create final training and test sets

```

```

63 X_train = X_filtered.loc[train_indices]
64 y_train = y.loc[train_indices]
65
66 X_test = X_filtered.loc[test_indices]
67 y_test = y.loc[test_indices]
68
69 print("Training set size:", X_train.shape[0])
70 print("Test set size:", X_test.shape[0])
71 print("Number of clusters:", n_clusters)

```

Visualizing the UMAP Clusters

A useful way to check the split is to plot the UMAP projection and color the points by cluster or by train/test assignment.

Listing 5.7: Visualizing UMAP clusters.

```

1  import matplotlib.pyplot as plt
2
3  plt.figure(figsize=(6, 5))
4  plt.scatter(
5      X_umap[:, 0],
6      X_umap[:, 1],
7      c=cluster_labels,
8      s=30
9  )
10
11 plt.xlabel("UMAP 1")
12 plt.ylabel("UMAP 2")
13 plt.title("UMAP Projection of Molecular Descriptor Space")
14 plt.show()

```

Why Cluster Splitting Can Be Useful

Cluster splitting can be more chemically meaningful than random splitting because it considers similarity between molecules. If the dataset contains several different chemical families, cluster splitting can help ensure that the model is evaluated across different regions of chemical space.

However, the choice of descriptors, UMAP parameters, number of clusters, and test-set selection strategy can affect the final split. Therefore, cluster splitting should be reported clearly in the methods section of a QSAR study.

5.4 Summary

Data cleaning and preprocessing are essential for reliable QSAR modeling. Invalid SMILES should be removed before descriptor calculation. Descriptor matrices should be checked for constant, near-constant, and highly correlated features. Finally, train/test splitting should be performed carefully because the splitting strategy strongly affects how model performance is interpreted.

Preprocessing Step	Purpose
Remove invalid SMILES	Prevent descriptor calculation errors
Remove constant descriptors	Remove features with no useful variation
Remove near-constant descriptors	Remove features with very little information
Remove highly correlated descriptors	Reduce redundancy and improve model stability
Random split	Simple baseline train/test split
Sorted split	Ensures target values are distributed across train and test sets
UMAP cluster split	Considers chemical-space distribution during splitting

Exercise

Using a small QSAR dataset, ask the student to complete the following tasks:

1. Remove invalid SMILES using RDKit.
2. Calculate molecular descriptors.
3. Remove constant and near-constant descriptors.
4. Remove descriptors with $|R| > 0.95$.
5. Perform a random split.
6. Perform a sorted split based on the target variable.
7. Perform a UMAP-based cluster split.
8. Compare the target-value ranges and dataset sizes for each split.

Chapter 6

Machine Learning Basics for QSAR

Learning Objectives

- Understand what a machine-learning model is.
- Explain the difference between rule-based systems and machine-learning models.
- Distinguish regression and classification problems in QSAR.
- Understand concepts of overfitting and underfitting.
- Understand why feature selection is important.
- Famous feature selection techniques in QSAR: Sequential feature selection and genetic algorithm.

Machine learning is one of the main tools used in modern QSAR modeling. In QSAR, machine-learning models learn relationships between molecular features, such as descriptors or fingerprints, and experimental endpoints, such as toxicity, solubility, or biological activity.

The goal of this chapter is not to cover all mathematical details of machine learning. Instead, the goal is to provide enough background so that students can understand what a model is, how models learn from data, how regression and classification are different, and why model validation and feature selection are important.

6.1 What Is a Model?

A **model** is a mathematical function that learns a relationship between input features and an output value.

In QSAR, the input features are usually molecular descriptors or fingerprints. These features describe the chemical structure numerically. The output is the endpoint we want to predict.

For a regression problem, the model can be written as:

$$\hat{y} = f(X)$$

where:

- X is the descriptor matrix,
- y is the experimental endpoint,
- $\hat{y}$ is the predicted endpoint,
- f is the model that learns the relationship between X and y .

For example, if we want to predict toxicity, the model learns how molecular properties such as molecular weight, logP, hydrogen bonding, charge distribution, and structural fragments are related to toxicity values.

Example in QSAR

Suppose we have a dataset containing three descriptors:

- molecular weight,
- logP,
- topological polar surface area.

The model uses these descriptors to predict an endpoint, such as LD₅₀, IC₅₀, EC₅₀, or solubility.

Molecular descriptors → Machine-learning model → Predicted activity

The model does not understand chemistry in the same way a human does. Instead, it identifies numerical patterns between molecular descriptors and the experimental endpoint.

6.2 Machine Learning vs. Rule-Based Systems

A **rule-based system** uses rules written directly by a human expert. In this approach, the expert decides what conditions should lead to a specific prediction.

For example, a simple rule-based toxicity system might say:

If molecular weight is high and logP is high, then the compound may have poor solubility or higher bioaccumulation potential.

In this case, the rules are manually defined by an expert.

In contrast, a **machine-learning model** learns the rules from data. Instead of the researcher manually writing every rule, the researcher provides examples of molecules and their experimental activities. The model then learns patterns from those examples.

The key difference is:

Rule-based system: Human defines the rules.

Machine learning: The model learns the rules from data.

Why Machine Learning Is Useful in QSAR

In QSAR, chemical activity can depend on many molecular properties at the same time. These relationships may be complex and difficult to describe using simple human-written rules.

Machine learning is useful because it can learn relationships involving many descriptors simultaneously. For example, a model may learn that toxicity depends on a combination of molecular size, hydrophobicity, polarity, electronic effects, and specific structural fragments.

Instructor Note

A useful explanation for students is that machine learning does not replace chemical knowledge. Instead, it helps detect patterns that may be difficult to manually define, and the researcher still needs to interpret whether those patterns make chemical sense.

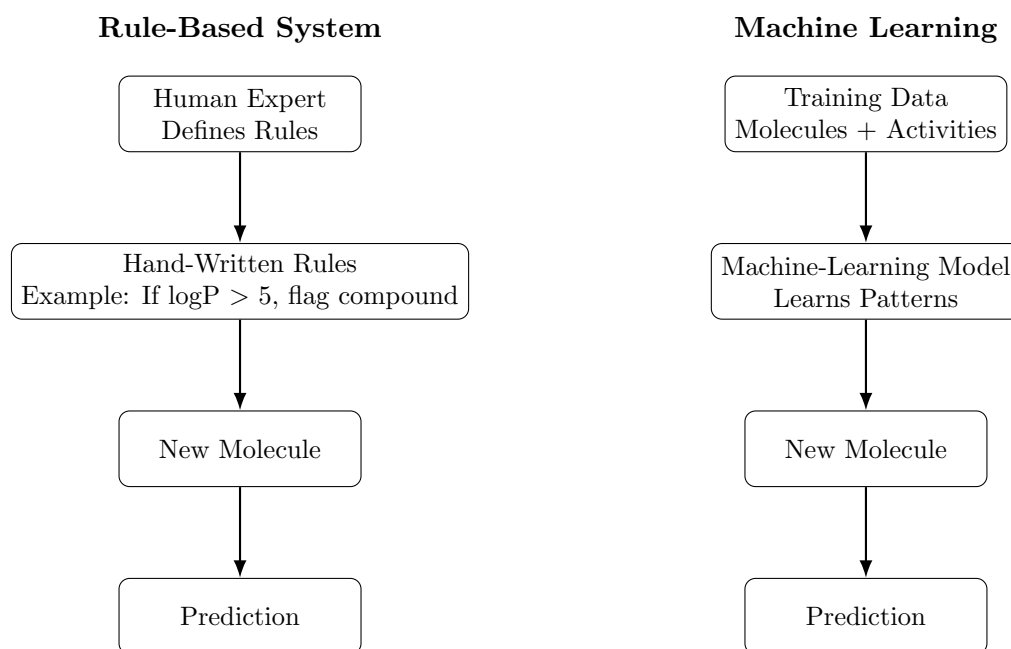


Figure 6.1: Comparison between a rule-based system and a machine-learning system. In a rule-based system, an expert manually defines the rules. In machine learning, the model learns patterns from examples in the training data.

6.3 Regression vs. Classification

QSAR models are usually divided into two main types: **regression models** and **classification models**.

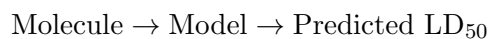
6.3.1 Regression

A **regression model** predicts a continuous numerical value.

Examples of regression endpoints include:

- LD₅₀,
- IC₅₀,
- EC₅₀,
- solubility,
- boiling point,
- melting point,
- binding affinity.

For example, if the goal is to predict the exact toxicity value of a compound, this is a regression problem.



6.3.2 Classification

A **classification model** predicts a category or class label.

Examples of classification endpoints include:

- toxic vs. non-toxic,
- active vs. inactive,
- inhibitor vs. non-inhibitor,
- soluble vs. poorly soluble,
- mutagenic vs. non-mutagenic.

For example, if the goal is to classify whether a compound is toxic or non-toxic, this is a classification problem.

Molecule $\rightarrow$ Model $\rightarrow$ Toxic or Non-Toxic

Regression and Classification Examples

QSAR Task	Output Type	Model Type
Predict LD ₅₀ value	Continuous number	Regression
Predict IC ₅₀ value	Continuous number	Regression
Predict solubility	Continuous number	Regression
Predict toxic vs. non-toxic	Category	Classification
Predict active vs. inactive	Category	Classification
Predict inhibitor vs. non-inhibitor	Category	Classification

Exercise

For each endpoint below, ask the student to decide whether the problem is regression or classification:

1. Predicting the exact IC₅₀ value of a compound.
2. Predicting whether a compound is active or inactive.
3. Predicting the aqueous solubility of a molecule.
4. Predicting whether a compound is mutagenic or non-mutagenic.
5. Predicting the LD₅₀ value of a chemical.

6.4 Overfitting and Underfitting

When building a QSAR model, the goal is not just to fit the training data well. The goal is to build a model that can make reliable predictions for new compounds.

Two common modeling problems are **overfitting** and **underfitting**.

6.4.1 Underfitting

Underfitting occurs when the model is too simple to learn the relationship between molecular descriptors and the endpoint.

An underfit model performs poorly on both the training set and the test set.

For example, a simple linear regression model may underfit if the true relationship between descriptors and toxicity is highly nonlinear.

6.4.2 Overfitting

Overfitting occurs when the model learns the training data too well, including noise or random patterns that are not meaningful.

An overfit model usually performs very well on the training set but poorly on the test set.

This is especially important in QSAR because datasets are often small, while the number of descriptors can be very large. For example, a dataset may contain only 100 molecules but thousands of descriptors. In this situation, a model may easily memorize the training compounds instead of learning general chemical patterns.

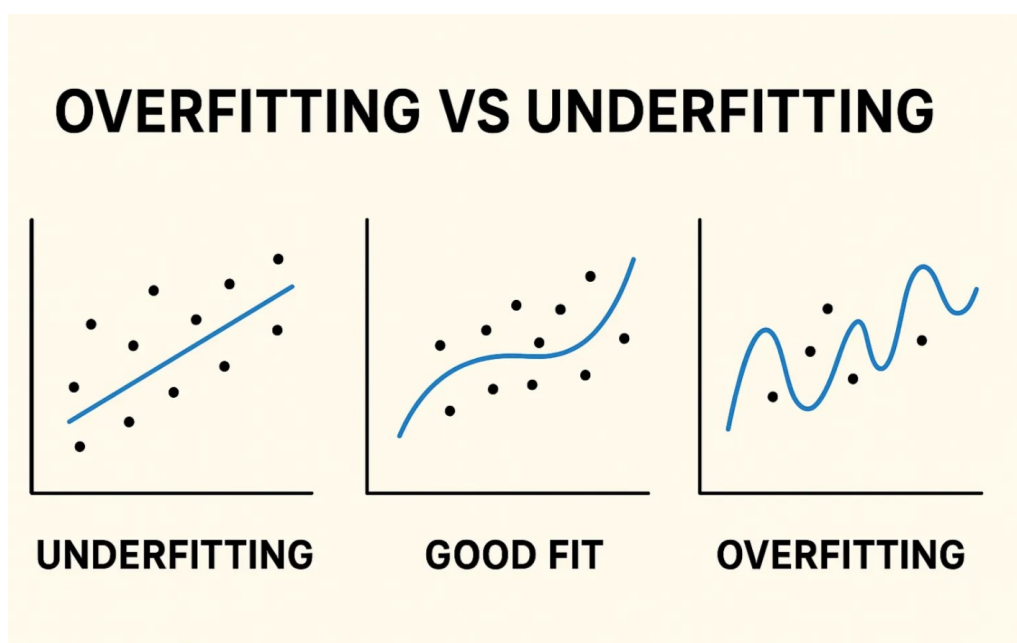


Figure 6.2: Underfitting vs. overfitting. A good fit finds the general trend of data and will give the most reliable prediction for an unseen datapoint.

6.4.3 Good Generalization

A good QSAR model should have reasonable performance on both the training set and the test set. The test-set performance is especially important because it estimates how well the model may perform on new molecules.

Situation	Training Performance	Test Performance	Interpretation
Underfitting	Poor	Poor	Model is too simple
Overfitting	Very good	Poor	Model memorized training data
Good fit	Good	Good	Model generalizes better

Instructor Note

A simple analogy is exam preparation. Underfitting is like not studying enough. Overfitting is like memorizing answers to practice questions without understanding the subject. A good model is like a student who understands the material and can answer new questions.

6.5 Feature Selection

Feature selection is the process of selecting a smaller set of useful descriptors from a larger descriptor pool.

In QSAR, feature selection is important because descriptor datasets can be very large. A single molecule may have hundreds or thousands of calculated descriptors. However, not all descriptors are useful for predicting the endpoint.

Some descriptors may be:

- irrelevant to the endpoint,
- redundant with other descriptors,
- noisy,
- difficult to interpret,
- harmful to model generalization.

Using too many descriptors can increase the risk of overfitting, especially when the dataset is small.

6.5.1 Why Feature Selection Is Important

Feature selection can help:

- reduce overfitting,
- improve model performance on external test sets,
- reduce model complexity,
- make models easier to interpret,
- remove redundant or noisy descriptors,
- decrease computational cost.

For example, if a QSAR dataset contains 150 molecules and 2,000 descriptors, the model has many more features than compounds. Without feature selection, the model may find random patterns that do not generalize to new molecules.

Large descriptor pool → Feature selection → Smaller useful descriptor set

6.5.2 Sequential Feature Selection

Sequential feature selection is a step-by-step method for selecting descriptors.

There are two common types:

- **Forward selection:** starts with no descriptors and adds one descriptor at a time.
- **Backward selection:** starts with all descriptors and removes one descriptor at a time.

In forward selection, the algorithm tests which descriptor improves the model the most and adds it to the selected set. Then it repeats the process by adding another descriptor. This continues until a stopping condition is reached, such as a selected number of descriptors or no further improvement in validation performance.

Forward Sequential Feature Selection Workflow

1. Start with an empty descriptor set.
2. Train models using each candidate descriptor.
3. Select the descriptor that gives the best validation performance.
4. Add that descriptor to the selected set.
5. Repeat the process by testing the remaining descriptors.
6. Stop when the desired number of descriptors is selected or performance no longer improves.

Listing 6.1: Example of sequential feature selection using scikit-learn.

```
1 from sklearn.feature_selection import SequentialFeatureSelector
2 from sklearn.ensemble import RandomForestRegressor
3 from sklearn.model_selection import KFold
4
5 # Define the model
6 model = RandomForestRegressor(
7     n_estimators=300,
8     random_state=42
9 )
10
11 # Define cross-validation
12 cv = KFold(
13     n_splits=5,
14     shuffle=True,
15     random_state=42
16 )
17
18 # Sequential feature selection
19 sfs = SequentialFeatureSelector(
20     estimator=model,
21     n_features_to_select=10,
22     direction="forward",
23     scoring="r2",
24     cv=cv,
25     n_jobs=-1
26 )
27
28 # Fit feature selector using training data only
```

```

29 sfs.fit(X_train, y_train)
30
31 # Selected descriptor names
32 selected_features = X_train.columns[sfs.get_support()]
33
34 print("Selected descriptors:")
35 print(selected_features.tolist())
36
37 # Create reduced train and test sets using the same selected
   descriptors
38 X_train_selected = X_train[selected_features]
39 X_test_selected = X_test[selected_features]

```

Important: Feature selection should be performed using only the training set. After the descriptors are selected from the training set, the same descriptors should be selected from the test set. The test set should not be used during feature selection.

6.5.3 Genetic Algorithm-Based Feature Selection

A **genetic algorithm**, often abbreviated as GA, is an optimization method inspired by natural selection. In QSAR, a genetic algorithm can be used to search for a good subset of descriptors.

Instead of adding or removing descriptors one at a time, a genetic algorithm starts with many possible descriptor subsets. Each subset is evaluated using model performance. The best subsets are selected, modified, and combined to produce new subsets.

Basic Genetic Algorithm Concepts

GA Term	Meaning in Feature Selection
Individual	One possible descriptor subset
Population	A group of descriptor subsets
Fitness	Model performance for a descriptor subset
Selection	Choosing better-performing subsets
Crossover	Combining parts of two descriptor subsets
Mutation	Randomly adding or removing descriptors
Generation	One cycle of selection, crossover, and mutation

Genetic Algorithm Workflow

1. Generate an initial population of random descriptor subsets.
2. Train and validate a model for each subset.
3. Assign a fitness score to each subset based on model performance.
4. Select the better-performing subsets.
5. Create new subsets using crossover and mutation.
6. Repeat the process for multiple generations.
7. Choose the descriptor subset with the best validation performance.

A simple way to think about GA feature selection is:

Many possible descriptor subsets → Evolutionary search → Best descriptor subset

Why Use a Genetic Algorithm?

Genetic algorithms can be useful when the descriptor space is large and many descriptor combinations are possible. They are able to search a wider range of possible feature subsets compared with simple sequential methods.

However, genetic algorithms can also be computationally expensive because many models must be trained during the search. The results may also depend on settings such as population size, mutation rate, crossover rate, and number of generations.

6.5.4 Sequential Feature Selection vs. Genetic Algorithm

Method	Advantages	Limitations
Sequential Feature Selection	Easy to understand; systematic; available in scikit-learn	Can be slow; may miss better descriptor combinations
Genetic Algorithm	Searches many possible combinations; useful for large descriptor spaces	More complex; computationally expensive; requires parameter tuning

6.6 Summary

Machine learning models learn relationships between molecular descriptors and experimental endpoints. Unlike rule-based systems, where an expert manually defines the rules, machine-learning models learn patterns from examples in the training data.

QSAR problems can be regression problems, where the goal is to predict a continuous value, or classification problems, where the goal is to predict a category. Model performance must be evaluated carefully to avoid underfitting and overfitting.

Feature selection is an important step in QSAR because molecular descriptor datasets often contain many irrelevant or redundant descriptors. Methods such as sequential feature selection and genetic algorithms can help identify smaller and more useful descriptor subsets.

Exercise

Ask the student to answer the following questions:

1. What is the main difference between a rule-based system and a machine-learning model?
2. Is predicting IC_{50} a regression or classification problem?
3. Is predicting active vs. inactive a regression or classification problem?
4. What is overfitting?
5. Why can too many descriptors be a problem in QSAR?
6. What is the difference between sequential feature selection and genetic algorithm-based feature selection?

Chapter 7

Internal and External Validation

Learning Objectives

- Understand the difference between internal and external validation.
- Explain why validation is necessary in QSAR modeling.
- Understand k-fold cross-validation
- Understand leave-one-out cross-validation.
- Explain the role of an external test set in evaluating QSAR models.

Validation is one of the most important parts of QSAR modeling. A model should not only fit the training data well; it should also make reliable predictions for new molecules. Validation methods help estimate how well a model may perform when applied to compounds that were not used to build it.

In QSAR, validation is often divided into two major categories:

- **Internal validation:** model performance is evaluated using only the training data, usually through resampling methods such as cross-validation.
- **External validation:** model performance is evaluated using an independent test set that was not used during model training.

Both types of validation are important. Internal validation helps assess the stability of the model during development, while external validation provides a more realistic estimate of prediction performance on new compounds.

7.1 Internal Validation

Internal validation refers to validation methods performed within the training dataset. The goal is to estimate how stable and reliable the model is without using the external test set.

Internal validation is useful because one train/test split may not be enough to fully evaluate a model. Depending on how the data are split, the model may appear better or worse. Cross-validation helps reduce this problem by repeatedly training and validating the model on different subsets of the training data.

Common internal validation methods include:

- k-fold cross-validation,
- leave-one-out cross-validation.

Important: Internal validation should be performed using only the training set. The external test set should remain untouched until final model evaluation.

7.1.1 K-Fold Cross-Validation

In **k-fold cross-validation**, the training data are divided into k approximately equal parts, called folds. The model is trained using $k - 1$ folds and validated on the remaining fold. This process is repeated k times so that each fold is used once as the validation fold.

For example, in **5-fold cross-validation**, the training data are divided into five folds:

Fold 1, Fold 2, Fold 3, Fold 4, Fold 5

The model is trained and validated five times:

Round	Training Folds	Validation Fold
1	Folds 2, 3, 4, 5	Fold 1
2	Folds 1, 3, 4, 5	Fold 2
3	Folds 1, 2, 4, 5	Fold 3
4	Folds 1, 2, 3, 5	Fold 4
5	Folds 1, 2, 3, 4	Fold 5

After the five rounds are complete, the validation scores are averaged. This average cross-validation score gives an estimate of how well the model performs across different subsets of the training data.

$$\text{Mean CV score} = \frac{\text{Score}_1 + \text{Score}_2 + \text{Score}_3 + \text{Score}_4 + \text{Score}_5}{5}$$

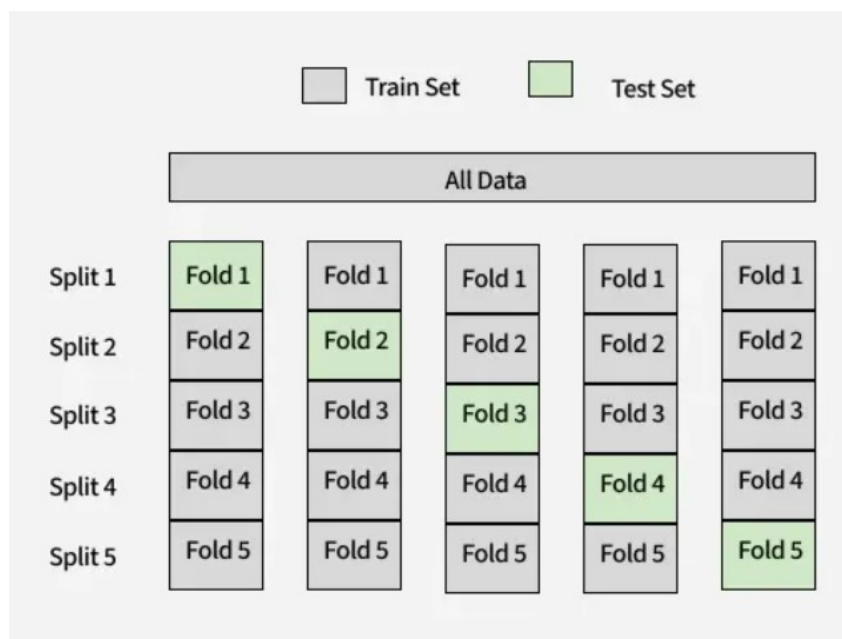


Figure 7.1: visualization of 5-fold cross-validation. Remember that this is an internal validation technique, meaning that we split the training data into 5 folds, we train the model on 4 folds, and we test on the fold we skip, then we repeat the process 5 times so that we have each fold as a test set. Once we make sure the model shows a good performance and stabilization over different folds, we can use it to predict the unseen test set.

For regression QSAR models, the score may be R^2 , MAE, or RMSE. For classification QSAR models, the score may be accuracy, F1-score, MCC, or ROC-AUC.

Listing 7.1: Example of 5-fold cross-validation for a regression QSAR model.

```

1 from sklearn.model_selection import KFold, cross_val_score
2 from sklearn.ensemble import RandomForestRegressor
3 import numpy as np
4
5 # Define the model
6 model = RandomForestRegressor(
7     n_estimators=300,
8     random_state=42
9 )
10
11 # Define 5-fold cross-validation
12 cv = KFold(
13     n_splits=5,
14     shuffle=True,
15     random_state=42
16 )
17
18 # Perform cross-validation using training data only
19 cv_scores = cross_val_score(
20     model,
21     X_train,
22     y_train,
23     cv=cv,
24     scoring="r2"
25 )
26
27 print("5-fold CV R2 scores:", cv_scores)
28 print("Mean CV R2:", np.mean(cv_scores))
29 print("Standard deviation:", np.std(cv_scores))

```

The mean cross-validation score shows the average internal performance of the model. The standard deviation shows how much the performance changes across different folds. A large standard deviation may suggest that the model is unstable or that the dataset is small or heterogeneous.

7.1.2 Leave-One-Out Cross-Validation

Leave-one-out cross-validation, often abbreviated as LOOCV, is a special case of k-fold cross-validation where the number of folds is equal to the number of training samples.

In LOOCV, one compound is left out as the validation sample, and the model is trained on all remaining compounds. This process is repeated until every compound has been used once as the validation sample.

If the training set contains n compounds, LOOCV trains the model n times.

For example, if a training set contains 100 compounds:

$$k = n = 100$$

This means the model is trained 100 times. Each time, 99 compounds are used for training and 1 compound is used for validation.

Round	Training Set	Validation Compound
1	All compounds except compound 1	Compound 1
2	All compounds except compound 2	Compound 2
3	All compounds except compound 3	Compound 3
...	...	...
n	All compounds except compound n	Compound n

LOOCV can be useful for small datasets because it uses almost all available data for training in each round. However, it can be computationally expensive for large datasets because the model must be trained many times.

Listing 7.2: Example of leave-one-out cross-validation.

```

1 from sklearn.model_selection import LeaveOneOut, cross_val_score
2 from sklearn.ensemble import RandomForestRegressor
3 import numpy as np
4
5 # Define the model
6 model = RandomForestRegressor(
7     n_estimators=300,
8     random_state=42
9 )
10
11 # Define leave-one-out cross-validation
12 loo = LeaveOneOut()
13
14 # Perform LOOCV using training data only
15 loo_scores = cross_val_score(
16     model,
17     X_train,
18     y_train,
19     cv=loo,
20     scoring="r2"
21 )
22
23 print("Number of LOOCV rounds:", len(loo_scores))
24 print("Mean LOOCV R2:", np.mean(loo_scores))

```

Note: In practice, R^2 may not always be meaningful for a single validation sample in each LOOCV round. Therefore, for regression tasks, it is often better to collect all LOOCV predictions and then calculate overall metrics such as R^2 , MAE, and RMSE.

Listing 7.3: LOOCV predictions followed by overall regression metrics.

```

1 from sklearn.model_selection import LeaveOneOut, cross_val_predict
2 from sklearn.ensemble import RandomForestRegressor
3 from sklearn.metrics import r2_score, mean_absolute_error,
4     mean_squared_error
5 import numpy as np
6
7 model = RandomForestRegressor(
8     n_estimators=300,
9     random_state=42
10 )
11
12 loo = LeaveOneOut()
13
14 # Generate cross-validated predictions

```

```

14 y_pred_loo = cross_val_predict(
15     model,
16     X_train,
17     y_train,
18     cv=loo
19 )
20
21 # Calculate overall LOOCV metrics
22 r2_loo = r2_score(y_train, y_pred_loo)
23 mae_loo = mean_absolute_error(y_train, y_pred_loo)
24 rmse_loo = np.sqrt(mean_squared_error(y_train, y_pred_loo))
25
26 print("LOOCV R2:", r2_loo)
27 print("LOOCV MAE:", mae_loo)
28 print("LOOCV RMSE:", rmse_loo)

```

7.1.3 Internal Validation Summary

Internal validation helps answer the question:

How stable is the model during training?

However, internal validation is not a replacement for external validation. Even if a model performs well during cross-validation, it still needs to be evaluated on an independent external test set.

7.2 External Validation

External validation evaluates the final model using a separate test set that was not used during model training or model selection.

The external test set should not be used for:

- descriptor filtering,
- feature selection,
- hyperparameter tuning,
- model comparison,
- scaling parameter estimation,
- cross-validation.

The purpose of external validation is to estimate how well the model performs on new compounds. In QSAR, this is especially important because the final goal is usually to predict activities for molecules that have not yet been experimentally tested.

7.2.1 External Test Set Workflow

A proper external validation workflow is:

1. Split the dataset into training and external test sets.
2. Use only the training set for preprocessing decisions, feature selection, model training, cross-validation, and hyperparameter tuning.

3. Train the final model using the selected training data and selected descriptors.
4. Apply the same preprocessing steps to the external test set.
5. Predict the endpoint values for the external test set.
6. Calculate external validation metrics.

Listing 7.4: Example of external validation after model training.

```
1 from sklearn.ensemble import RandomForestRegressor
2 from sklearn.metrics import r2_score, mean_absolute_error,
   mean_squared_error
3 import numpy as np
4
5 # Train the final model using the training set
6 final_model = RandomForestRegressor(
7     n_estimators=300,
8     random_state=42
9 )
10
11 final_model.fit(X_train_selected, y_train)
12
13 # Predict the external test set
14 y_pred_test = final_model.predict(X_test_selected)
15
16 # Calculate external validation metrics
17 r2_ext = r2_score(y_test, y_pred_test)
18 mae_ext = mean_absolute_error(y_test, y_pred_test)
19 rmse_ext = np.sqrt(mean_squared_error(y_test, y_pred_test))
20
21 print("External test R2:", r2_ext)
22 print("External test MAE:", mae_ext)
23 print("External test RMSE:", rmse_ext)
```

7.2.2 Why External Validation Is Important

External validation is important because it provides a more realistic estimate of model performance. A model may perform well during internal validation but still fail on truly new compounds.

This can happen when:

- the dataset is small,
- the model is overfit,
- the test compounds are chemically different from the training compounds,
- feature selection was not performed correctly,
- data leakage occurred during preprocessing,
- the endpoint contains experimental noise.

A strong QSAR model should show good performance in both internal and external validation.

7.2.3 Internal vs. External Validation

Validation Type	Data Used	Purpose
Internal validation	Training data only	Estimates model stability during development
External validation	Independent test set	Estimates performance on new compounds

7.2.4 Common Mistakes

Common validation mistakes in QSAR include:

- using the test set during feature selection,
- scaling the full dataset before train/test splitting,
- removing highly correlated descriptors using the full dataset,
- tuning hyperparameters based on test-set performance,
- reporting only training performance,
- reporting cross-validation performance without external validation.

These mistakes can lead to overly optimistic model performance. To avoid this, the external test set should remain independent until the final evaluation.

7.3 Summary

Internal and external validation are both essential in QSAR modeling. Internal validation, such as 5-fold cross-validation or leave-one-out cross-validation, helps evaluate model stability during development. External validation evaluates the final model on an independent test set and provides a more realistic estimate of prediction performance.

A reliable QSAR model should not only perform well during internal validation but also show acceptable performance on external validation.

Exercise

Ask the student to answer the following questions:

1. What is the difference between internal and external validation?
2. In 5-fold cross-validation, how many times is the model trained?
3. In leave-one-out cross-validation, how many validation compounds are used in each round?
4. Why should the external test set not be used during feature selection?
5. Which validation result is more important for estimating performance on new molecules: training performance, cross-validation performance, or external test performance?

Chapter 8

Applicability Domain

Learning Objectives

- Define applicability domain in QSAR modeling.
- Explain why applicability domain is important for reliable prediction.
- Describe common approaches for defining applicability domain.
- Understand distance-based, similarity-based, and Williams plot approaches.
- Explain the x-axis and y-axis of a Williams plot.
- Interpret compounds with high leverage or standardized residuals greater than ± 3 .

A QSAR model is usually developed using a limited set of molecules. Therefore, the model is expected to make reliable predictions only for compounds that are similar to the molecules used during training. This concept is called the **applicability domain**, often abbreviated as **AD**.

8.1 What Is Applicability Domain?

The applicability domain defines **the chemical space where a QSAR model is considered reliable**. In other words, it describes the region of chemical structures and descriptor values where the model has enough training information to make meaningful predictions.

A QSAR model should not be assumed to work equally well for every possible molecule. For example, if a model is trained only on small organic molecules, it may not make reliable predictions for very large molecules, polymers, metal complexes, or molecules with chemical features not present in the training set.

The applicability domain answers the question: *Is this new molecule similar enough to the training compounds for the model prediction to be trusted?*

8.1.1 Why Applicability Domain Is Important

Applicability domain is important because a model may produce a numerical prediction for any molecule, even when that prediction is not reliable. Machine-learning models usually do not automatically know when they are being applied outside their reliable chemical space.

For example, a QSAR model may predict the toxicity of a molecule that is very different from all training compounds. **The prediction may look reasonable as a number, but it**

may be scientifically unreliable because the model has never seen similar chemistry during training.

Applicability domain helps identify:

- compounds that are structurally similar to the training set,
- compounds that are very different from the training set,
- predictions that may be unreliable,
- possible outliers in the training or test set,
- chemical regions where more experimental data may be needed.

In QSAR reporting, applicability domain is important because it helps prevent overinterpretation of model predictions. A good QSAR model should report not only predicted values but also whether each prediction falls inside or outside the model’s applicability domain.

8.2 Approaches for Defining Applicability Domain

Several methods can be used to define applicability domain. Different methods use different ideas of molecular similarity, descriptor space, or prediction error.

Common approaches include:

- distance-based applicability domain,
- similarity-based applicability domain,
- leverage-based applicability domain using the Williams plot.

8.2.1 Distance-Based Applicability Domain

Distance-based methods define applicability domain based on how far a compound is from the training compounds in descriptor space.

In this approach, each molecule is represented by its molecular descriptors. A new compound is considered inside the applicability domain if it is close enough to the training compounds.

Common distance measures include:

- Euclidean distance,
- Mahalanobis distance,
- distance to nearest training compound,
- distance to the centroid of the training set.

For example, if a new molecule has descriptor values very far from the training molecules, it may be considered outside the applicability domain.

$$d_i = \sqrt{(x_i - x_j)^2}$$

where d_i represents distance in descriptor space.

For multiple descriptors, Euclidean distance can be written as:

$$d(i, j) = \sqrt{\sum_{k=1}^p (x_{ik} - x_{jk})^2}$$

where:

- x_{ik} is the value of descriptor k for molecule i ,
- x_{jk} is the value of descriptor k for molecule j ,
- p is the number of descriptors.

Interpretation

A molecule close to the training compounds is more likely to be inside the applicability domain. A molecule far from the training compounds may be outside the applicability domain and its prediction should be treated with caution.

Distance Result	Interpretation
Small distance to training compounds	More likely inside AD
Large distance to training compounds	More likely outside AD

8.2.2 Similarity-Based Applicability Domain

Similarity-based methods define applicability domain based on molecular similarity to the training set. These methods are commonly used when molecules are represented by fingerprints, such as Morgan fingerprints.

One common similarity measure is the **Tanimoto similarity**. It is often used to compare molecular fingerprints.

Tanimoto similarity ranges from 0 to 1:

- 1 means two molecules are identical based on their fingerprints,
- 0 means two molecules have no fingerprint features in common.

A new molecule can be considered inside the applicability domain if it has sufficient similarity to at least one training molecule.

For example:

Inside AD if maximum Tanimoto similarity to training set ≥ 0.6

The exact threshold depends on the dataset and the modeling task.

Interpretation

If a test molecule is highly similar to one or more training molecules, the model is more likely to make a reliable prediction. If the molecule has low similarity to all training molecules, the prediction may be unreliable.

Similarity Result	Interpretation
High similarity to training molecules	More likely inside AD
Low similarity to all training molecules	More likely outside AD

8.2.3 Williams Plot

The **Williams plot** is one of the most common applicability domain methods in QSAR. It is especially common in descriptor-based QSAR modeling.

A Williams plot shows two important pieces of information:

- the **leverage** of each compound on the x-axis,

- the **standardized residual** of each compound on the y-axis.

The Williams plot helps identify:

- compounds that are structurally far from the training set,
- compounds with unusually large prediction errors,
- possible response outliers,
- compounds outside the model’s applicability domain.

8.3 Understanding the Williams Plot

8.3.1 X-Axis: Leverage

The x-axis of a Williams plot is the **leverage** value, usually written as h_i .

Leverage measures how far a compound is from the center of the training descriptor space. A compound with high leverage has descriptor values that are unusual compared with most training compounds.

For the training set, leverage is calculated from the hat matrix:

$$H = X(X^T X)^{-1} X^T$$

where:

- H is the hat matrix,
- X is the descriptor matrix,
- X^T is the transpose of the descriptor matrix,
- $(X^T X)^{-1}$ is the inverse of $X^T X$.

The leverage value for compound i is the i th diagonal element of the hat matrix:

$$h_i = H_{ii}$$

For a new or test compound, leverage can be calculated as:

$$h_i = x_i^T (X^T X)^{-1} x_i$$

where x_i is the descriptor vector of the new compound.

8.3.2 Warning Leverage Threshold

A compound is usually considered to have high leverage if its leverage is greater than the warning leverage value, h^* .

The warning leverage is commonly calculated as:

$$h^* = \frac{3(p+1)}{n}$$

where:

- p is the number of model descriptors,
- n is the number of training compounds,

- $p + 1$ includes the intercept term.

If a compound has:

$$h_i > h^*$$

then it is considered structurally influential or outside the main descriptor space of the training set.

Interpretation of High Leverage

A high-leverage compound is not automatically a bad compound. It means the compound is chemically or structurally different from the majority of the training set.

For training compounds, high leverage means the compound may strongly influence the model.

For test compounds, high leverage means the compound may be outside the applicability domain, and its prediction should be interpreted carefully.

8.3.3 Y-Axis: Standardized Residual

The y-axis of a Williams plot is the **standardized residual**.

A residual is the difference between the experimental value and the predicted value:

$$e_i = y_i - \hat{y}_i$$

where:

- e_i is the residual for compound i ,
- y_i is the experimental value,
- $\hat{y}_i$ is the predicted value.

The standardized residual is calculated by dividing the residual by the standard deviation of residuals:

$$r_i = \frac{e_i}{s}$$

where:

- r_i is the standardized residual,
- e_i is the residual,
- s is the residual standard deviation.

The residual standard deviation can be estimated from the training set as:

$$s = \sqrt{\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n - p - 1}}$$

where:

- n is the number of training compounds,
- p is the number of descriptors used in the model.

8.3.4 Meaning of ± 3 Standardized Residuals

In a Williams plot, horizontal lines are usually drawn at:

$$r_i = +3$$

and

$$r_i = -3$$

A compound with standardized residual greater than $+3$ or less than -3 has a much larger prediction error than expected.

$$|r_i| > 3$$

This means the compound may be a **response outlier**. In other words, the compound's experimental activity is not well predicted by the model.

Interpretation of Large Standardized Residuals

If a compound has $|r_i| > 3$, this may indicate:

- possible experimental error,
- unusual biological behavior,
- missing important descriptors,
- an incorrect molecular structure,
- a compound that follows a different mechanism of action,
- poor model prediction for that compound.

A large standardized residual does not automatically mean the compound should be removed. It means the compound should be investigated carefully.

8.4 Interpreting Regions of the Williams Plot

The Williams plot can be divided into different regions based on leverage and standardized residuals.

Region	Interpretation
$h_i \leq h^*$ and $ r_i \leq 3$	Compound is inside the applicability domain and has acceptable prediction error.
$h_i > h^*$ and $ r_i \leq 3$	Compound has high leverage but acceptable prediction error. It may be structurally influential or outside the main chemical space.
$h_i \leq h^*$ and $ r_i > 3$	Compound has normal leverage but large prediction error. It may be a response outlier.
$h_i > h^*$ and $ r_i > 3$	Compound has high leverage and large prediction error. It is likely outside the applicability domain and should be investigated carefully.

8.4.1 Inside vs. Outside Applicability Domain

A common rule is:

- A compound is considered **inside the applicability domain** if $h_i \leq h^*$ and $|r_i| \leq 3$.
- A compound may be considered **outside the applicability domain** if $h_i > h^*$ or $|r_i| > 3$.

However, interpretation should be done carefully. A compound with high leverage but low residual may still be predicted well. A compound with normal leverage but high residual may indicate experimental noise or missing chemical information.

8.5 Example Python Code for Williams Plot

The following code shows how to calculate leverage, standardized residuals, and the warning leverage threshold for a regression QSAR model.

Listing 8.1: Calculating leverage and standardized residuals for a Williams plot.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.ensemble import RandomForestRegressor
6
7 # X_train_selected and X_test_selected contain the selected descriptors
8 # y_train and y_test contain the experimental endpoint values
9
10 # Scale descriptors using training set only
11 scaler = StandardScaler()
12
13 X_train_scaled = scaler.fit_transform(X_train_selected)
14 X_test_scaled = scaler.transform(X_test_selected)
15
16 # Train a regression model
17 model = RandomForestRegressor(
18     n_estimators=300,
19     random_state=42
20 )
21
22 model.fit(X_train_scaled, y_train)
23
24 # Predict training and test sets
25 y_train_pred = model.predict(X_train_scaled)
26 y_test_pred = model.predict(X_test_scaled)
27
28 # Number of training compounds and number of descriptors
29 n_train = X_train_scaled.shape[0]
30 p = X_train_scaled.shape[1]
31
32 # Add intercept column for leverage calculation
33 X_train_aug = np.column_stack([
34     np.ones(n_train),
35     X_train_scaled
36 ])
37
38 X_test_aug = np.column_stack([
39     np.ones(X_test_scaled.shape[0]),
```

```

40     X_test_scaled
41 ])
42
43 # Calculate leverage values
44 # For training compounds
45 hat_matrix_train = (
46     X_train_aug
47     @ np.linalg.pinv(X_train_aug.T @ X_train_aug)
48     @ X_train_aug.T
49 )
50
51 leverage_train = np.diag(hat_matrix_train)
52
53 # For test compounds
54 hat_matrix_test = (
55     X_test_aug
56     @ np.linalg.pinv(X_train_aug.T @ X_train_aug)
57     @ X_test_aug.T
58 )
59
60 leverage_test = np.diag(hat_matrix_test)
61
62 # Calculate residuals
63 residuals_train = y_train - y_train_pred
64 residuals_test = y_test - y_test_pred
65
66 # Calculate residual standard deviation from training set
67 s = np.sqrt(
68     np.sum(residuals_train ** 2) / (n_train - p - 1)
69 )
70
71 # Calculate standardized residuals
72 std_res_train = residuals_train / s
73 std_res_test = residuals_test / s
74
75 # Warning leverage threshold
76 h_star = 3 * (p + 1) / n_train
77
78 print("Warning leverage h*:", h_star)

```

The Williams plot can then be drawn using the leverage values on the x-axis and standardized residuals on the y-axis.

Listing 8.2: Drawing a Williams plot.

```

1 plt.figure(figsize=(7, 5))
2
3 # Training compounds
4 plt.scatter(
5     leverage_train,
6     std_res_train,
7     label="Training compounds",
8     alpha=0.7
9 )
10
11 # Test compounds
12 plt.scatter(
13     leverage_test,
14     std_res_test,

```

```

15     label="Test compounds",
16     alpha=0.7
17 )
18
19 # Horizontal lines at standardized residuals = +3 and -3
20 plt.axhline(y=3, linestyle="--")
21 plt.axhline(y=-3, linestyle="--")
22
23 # Vertical line at warning leverage
24 plt.axvline(x=h_star, linestyle="--")
25
26 plt.xlabel("Leverage")
27 plt.ylabel("Standardized residual")
28 plt.title("Williams Plot")
29 plt.legend()
30 plt.show()

```

Instructor Note

Although the Williams plot was originally developed for linear descriptor-based models, it is commonly used in QSAR as a practical way to evaluate chemical space and response outliers. For nonlinear models, leverage should be interpreted as a descriptor-space diagnostic rather than a complete measure of model uncertainty.

8.6 Comparison of Applicability Domain Methods

AD Method	Main Idea	Useful For
Distance-based AD	Measures distance from training compounds in descriptor space	Detecting compounds far from the training descriptor distribution
Similarity-based AD	Measures molecular similarity to training compounds, often using fingerprints	Fingerprint-based models and structural similarity analysis
Williams plot	Uses leverage and standardized residuals	Identifying high-leverage compounds and response outliers

8.7 Summary

Applicability domain defines the region of chemical space where a QSAR model is expected to make reliable predictions. It is important because machine-learning models can generate predictions even for molecules that are very different from the training set.

Distance-based methods evaluate how far a compound is from the training molecules in descriptor space. Similarity-based methods evaluate how similar a compound is to the training molecules, often using molecular fingerprints. The Williams plot uses leverage and standardized residuals to identify compounds that are outside the model's descriptor space or have unusually large prediction errors.

In a Williams plot, the x-axis represents leverage, which measures how far a compound is from the training descriptor space. The y-axis represents standardized residuals, which measure prediction error. Compounds with leverage greater than the warning leverage h^* may be structurally influential or outside the applicability domain. Compounds with standardized residuals greater than ± 3 may be response outliers.

Exercise

Ask the student to answer the following questions:

1. What is applicability domain?
2. Why should a QSAR prediction outside the applicability domain be treated carefully?
3. What is the difference between distance-based and similarity-based applicability domain?
4. What does the x-axis of a Williams plot represent?
5. What does the y-axis of a Williams plot represent?
6. What does it mean if a compound has leverage greater than h^* ?
7. What does it mean if a compound has a standardized residual greater than +3 or less than -3?

Chapter 9

Interpreting and Explaining a QSAR Model

Learning Objectives

- Explain why model interpretability is important in QSAR.
- Understand how molecular descriptors influence model predictions.
- Distinguish positive, negative, and nonlinear descriptor effects.
- Understand the basic idea of SHAP values.
- Understand the basic idea of Accumulated Local Effects.
- Interpret model explanations using chemical reasoning.

Building a QSAR model is not only about making accurate predictions. In many scientific applications, we also want to understand **why** the model makes a prediction. This is called **model interpretation** or **model explanation**.

In QSAR modeling, interpretation is especially important because the model should help connect molecular structure to biological or physicochemical activity. A useful QSAR model should not only predict toxicity, solubility, or activity, but also provide insight into which molecular features are associated with the endpoint.

9.1 Why Interpretability Is Important

QSAR models are often used in scientific decision-making. Therefore, it is important to understand what the model has learned from the data. If a model gives good statistical performance but cannot be explained chemically, it may be difficult to trust or use in practice.

Interpretability helps answer questions such as:

- Which descriptors are most important for prediction?
- Does increasing a descriptor increase or decrease the predicted endpoint?
- Are the relationships between descriptors and the endpoint linear or nonlinear?
- Are the model’s learned patterns chemically reasonable?
- Could the model be relying on artifacts or dataset bias?
- Which molecular features may be responsible for high or low activity?

For example, if a toxicity model shows that increasing hydrophobicity is associated with increased toxicity, this may be chemically reasonable for some endpoints because hydrophobic compounds may interact more strongly with biological membranes. However, this relationship may not apply to every dataset or every biological system. Therefore, interpretation must always be connected to chemical and biological knowledge.

Instructor Note

Emphasize to students that interpretation does not automatically prove causation. A descriptor may be important because it is statistically associated with the endpoint, but additional chemical reasoning or experimental evidence is needed before making strong mechanistic claims.

9.2 Understanding Descriptor Effects

In QSAR, each molecule is represented by descriptors or fingerprints. When interpreting a model, we are often interested in understanding how each descriptor affects the predicted endpoint.

For example, suppose a model predicts toxicity. We may want to know:

- Does toxicity increase as molecular weight increases?
- Does toxicity decrease as polarity increases?
- Does logP have a linear or nonlinear effect?
- Is there an optimal range of a descriptor where activity is highest?
- Are some descriptors important only for certain chemical classes?

9.2.1 Positive Descriptor Effect

A descriptor has a **positive effect** when increasing the descriptor value tends to increase the predicted endpoint.

For example, if higher MolLogP values lead to higher predicted toxicity, then MolLogP has a positive effect on the model prediction.

Descriptor increases $\Rightarrow$ Predicted endpoint increases

9.2.2 Negative Descriptor Effect

A descriptor has a **negative effect** when increasing the descriptor value tends to decrease the predicted endpoint.

For example, if higher topological polar surface area, or TPSA, leads to lower predicted membrane permeability, then TPSA has a negative effect on the prediction.

Descriptor increases $\Rightarrow$ Predicted endpoint decreases

9.2.3 Nonlinear Descriptor Effect

Not all descriptor effects are linear. A descriptor may have a **nonlinear effect**, meaning the relationship changes across the descriptor range.

For example, very low logP may reduce membrane interaction, moderate logP may increase biological activity, and very high logP may reduce solubility. In this case, the effect of logP is not simply positive or negative across the full range.

Descriptor effect may depend on descriptor range

Nonlinear relationships are common in QSAR because biological activity often depends on a balance of molecular properties such as size, hydrophobicity, polarity, charge, and shape.

9.2.4 Descriptor Effects Are Model-Dependent

Descriptor interpretation depends on the model, dataset, endpoint, and chemical space. The same descriptor may have different effects in different QSAR models.

For example, molecular weight may be positively associated with one endpoint but negatively associated with another. Therefore, descriptor effects should always be interpreted in the context of the specific dataset and endpoint.

9.3 Feature Importance

Many machine-learning models can provide feature importance values. Feature importance gives a general ranking of descriptors based on how useful they are for prediction.

For example, a Random Forest model can estimate how much each descriptor contributes to reducing prediction error across decision trees.

Listing 9.1: Extracting feature importance from a Random Forest model.

```
1 import pandas as pd
2
3 # model is a trained Random Forest model
4 # X_train_selected contains the descriptors used for training
5
6 importance_df = pd.DataFrame({
7     "Descriptor": X_train_selected.columns,
8     "Importance": model.feature_importances_
9 })
10
11 importance_df = importance_df.sort_values(
12     by="Importance",
13     ascending=False
14 )
15
16 print(importance_df.head(10))
```

Feature importance is useful as a first step, but it has limitations. It usually tells us which descriptors are important, but not exactly how they affect the prediction. For example, feature importance may show that MolLogP is important, but it does not directly tell us whether increasing MolLogP increases or decreases predicted toxicity.

To better understand descriptor effects, methods such as SHAP values and Accumulated Local Effects can be used.

9.4 SHAP Values

SHAP stands for **SHapley Additive exPlanations**. SHAP is a model explanation method that estimates how much each descriptor contributes to a model prediction.

For a single molecule, SHAP values explain how each descriptor pushes the prediction higher or lower relative to a baseline prediction.

A simplified SHAP explanation can be written as:

$$\hat{y}_i = \phi_0 + \sum_{j=1}^p \phi_{ij}$$

where:

- $\hat{y}_i$ is the model prediction for compound i ,
- ϕ_0 is the baseline prediction,
- ϕ_{ij} is the SHAP value of descriptor j for compound i ,
- p is the number of descriptors.

9.4.1 Interpreting SHAP Values

A SHAP value can be positive or negative.

- A **positive SHAP value** means the descriptor increases the predicted endpoint.
- A **negative SHAP value** means the descriptor decreases the predicted endpoint.
- A SHAP value close to zero means the descriptor has little effect on that specific prediction.

For example, if MolLogP has a positive SHAP value for a compound in a toxicity model, this means MolLogP pushed the prediction toward higher toxicity for that compound.

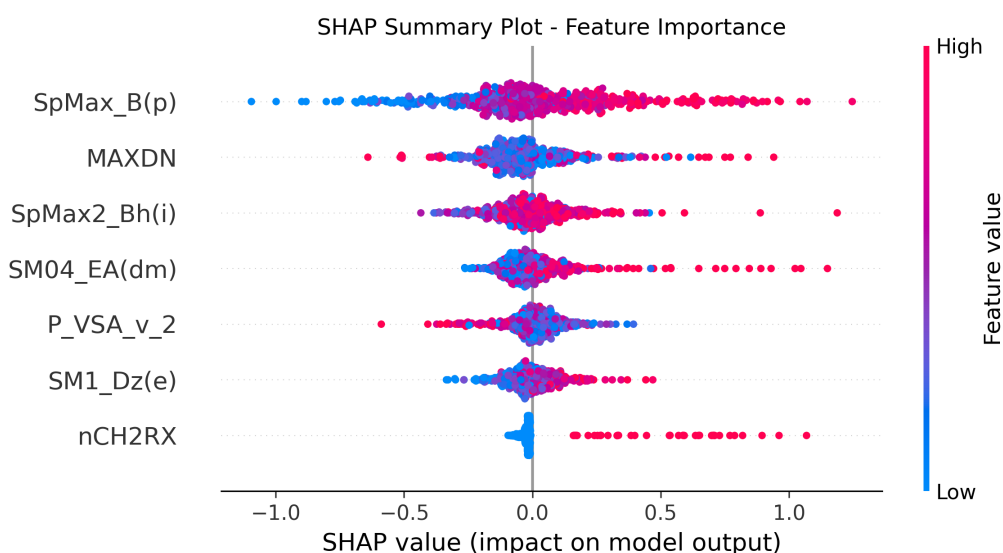


Figure 9.1: Example of SHAP value Beeswarm plot for a model with 7 descriptors.

In Figure 8.1, as an example in the case of SpMax_B(P), we can see that by the value of this descriptor (blue to red color), the impact of this descriptor becomes more positive. So, lower values show a negative effect and higher values show a positive effect.

9.5 Accumulated Local Effects

Accumulated Local Effects, often abbreviated as **ALE**, is another method for interpreting machine-learning models. ALE plots show how the model prediction changes as a descriptor changes.

ALE is useful for understanding whether a descriptor has:

- a positive relationship with the endpoint,
- a negative relationship with the endpoint,
- a nonlinear relationship with the endpoint,
- a threshold effect,
- an optimal range.

9.5.1 What Does an ALE Plot Show?

An ALE plot usually has:

- the descriptor value on the x-axis,
- the accumulated effect on the model prediction on the y-axis.

The x-axis shows the range of a descriptor, such as MolLogP or TPSA. The y-axis shows how the model prediction changes relative to the average effect.

9.5.2 Interpreting ALE Plots

An ALE curve can be interpreted as follows:

ALE Pattern	Interpretation
Increasing curve	Higher descriptor values increase the prediction
Decreasing curve	Higher descriptor values decrease the prediction
Flat curve	Descriptor has little effect on the prediction
Curved or changing slope	Descriptor has a nonlinear effect
Peak or valley	Descriptor may have an optimal range or threshold effect

Figure 8.1 shows an example of ALE plots. In order to interpret the effect of each descriptor on the endpoint (toxicity in this case), we need to look at the slope of the diagram. For example, in the case of nHM descriptor (number of heavy atoms), the ALE plot tells us that with an increase in the nHM value (in other words, an increasing number of heavy atoms) the toxicity will decrease because the slope of the diagram is negative.

Instructor Note

Ask your student to interpret the rest of the ALE plots in Figure 8.2.

9.5.3 ALE vs. Simple Correlation

ALE is different from simple correlation. A correlation coefficient only measures a general linear relationship between a descriptor and the endpoint. ALE uses the trained model to show how the model prediction changes across different values of a descriptor.

This is useful because many QSAR models, such as Random Forest, Gradient Boosting, XGBoost, and CatBoost, can learn nonlinear relationships.

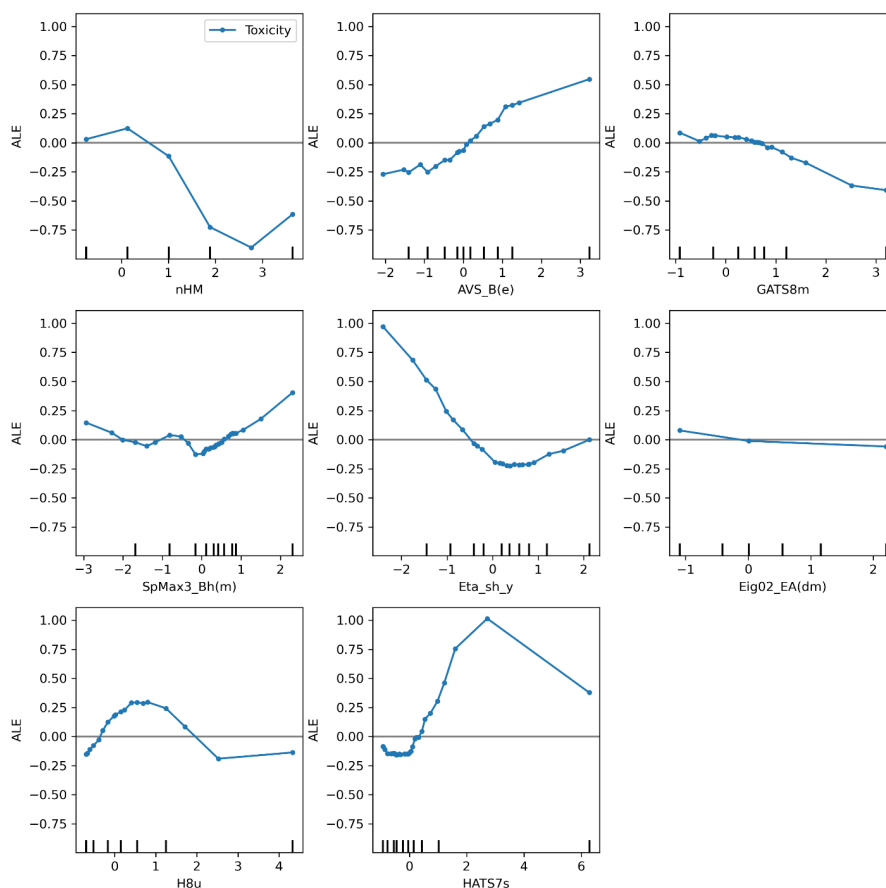


Figure 9.2: Example of ALE interpretation for a model with 8 descriptors (each plot is for one descriptor).

9.5.4 Example ALE Code

The following example shows how to generate an ALE plot for one descriptor.

Listing 9.2: Example of an Accumulated Local Effects plot.

```

1 # Install PyALE if needed:
2 # pip install PyALE
3
4 from PyALE import ale
5 import matplotlib.pyplot as plt
6
7 # model is a trained regression model
8 # X_train_selected contains the descriptors used for training
9
10 descriptor_name = "MolLogP"
11
12 ale_eff = ale(
13     X=X_train_selected,
14     model=model,
15     feature=[descriptor_name],
16     grid_size=20
17 )
18
19 plt.title("ALE Plot for " + descriptor_name)
20 plt.xlabel(descriptor_name)

```

```
21 plt.ylabel("Accumulated local effect")
22 plt.show()
```

9.5.5 Using ALE for QSAR Interpretation

In QSAR, ALE plots can help identify descriptor–endpoint relationships. For example:

- An increasing ALE curve for MolLogP may suggest that hydrophobicity increases the predicted endpoint.
- A decreasing ALE curve for TPSA may suggest that polarity decreases the predicted endpoint.
- A nonlinear ALE curve for molecular weight may suggest that activity increases only within a certain molecular-size range.

These trends should be interpreted chemically and compared with known mechanisms, literature, or experimental observations.

9.6 Important Cautions

Model interpretation should be done carefully. A model explanation shows what the model has learned from the dataset, but it does not automatically prove a chemical mechanism.

Important cautions include:

- Descriptor importance does not prove causation.
- Correlated descriptors can make interpretation more difficult.
- A descriptor may be important because it is correlated with another chemical property.
- Interpretation is only meaningful within the model’s applicability domain.
- Poor-quality data can lead to misleading explanations.
- Chemical reasoning is still necessary.

For example, if a model identifies molecular weight as important, this does not necessarily mean molecular weight itself causes toxicity. Molecular weight may be correlated with hydrophobicity, size, number of atoms, or other structural properties.

9.7 Summary

Interpreting and explaining a QSAR model helps connect machine-learning predictions to chemical understanding. Interpretability is important because QSAR models are often used for scientific reasoning, compound prioritization, and hypothesis generation.

In QSAR interpretation, we are often interested in how each descriptor affects the predicted endpoint. A descriptor may increase the prediction, decrease the prediction, or show a nonlinear relationship with the endpoint.

SHAP values explain how descriptors contribute to model predictions. They can be used for both global interpretation across the dataset and local interpretation for individual molecules. Accumulated Local Effects plots show how model predictions change as a descriptor value changes, making them useful for identifying positive, negative, and nonlinear descriptor effects.

A reliable interpretation should combine statistical explanation methods with chemical knowledge.

Appendix A

Example Dataset Template (To be updated/placeholder)

A simple CSV file for the tutorial can use the following format:

CompoundID	SMILES	Activity
Cmpd001	CCO	1.25
Cmpd002	CC(=O)O	0.95
Cmpd003	c1ccccc1	2.80

Appendix B

Suggested Student Report Template

Title

A short title describing the QSAR task.

Introduction

What endpoint was predicted and why is it important?

Dataset

Where did the data come from? How many compounds were used? What cleaning steps were performed?

Methods

What descriptors or fingerprints were calculated? What models were trained? How was validation performed?

Results

Include a model performance table and predicted vs. experimental plot.

Discussion

Which model performed best? Was there evidence of overfitting? Which descriptors were important? Are the results chemically meaningful?

Limitations

Describe limitations such as dataset size, endpoint variability, applicability domain, or lack of external validation.

Conclusion

Summarize the main result in two or three sentences.

Appendix C

Useful Resources

- RDKit documentation: <https://www.rdkit.org/docs/>
- scikit-learn documentation: <https://scikit-learn.org/stable/>
- pandas documentation: <https://pandas.pydata.org/docs/>
- DeepChem tutorials: <https://deepchem.io/>